CHAPTER 1
INTRODUCTION

1.1 Project Description

The last decade has seen social media grow from a niche communication tool into one of the most influential forces shaping public discourse commercial activity and political opinion worldwide. Platforms like Twitter and Instagram now host hundreds of millions of active users and that scale has made them attractive not just to genuine people but to those who want to exploit the reach of these networks for deceptive purposes. The creation of fake profiles automated bot accounts and AI-generated face images has grown into a structured problem that affects platform integrity election credibility consumer trust and individual safety [2 24].

Fake accounts on social media are not a new phenomenon but the methods used to create and operate them have become considerably more advanced over time. Early fake accounts were often easy to identify — they had no profile picture minimal followers and posted repetitive content at unnatural intervals. Today the situation is different. Bot operators have learned to mimic organic growth patterns use stolen or AI-generated photographs as profile pictures and construct believable backstories through carefully written bios [4 31]. This evolution has made manual detection impractical at scale and has pushed researchers toward automated machine learning-based solutions [1 3].

Detection research in this space has followed a broadly similar trajectory across different platforms. On Twitter early work focused on account metadata — follower counts tweet frequency account age and the ratio of original posts to retweets [16 18]. Systems like BotOrNot developed by Varol et al. [19] demonstrated that combining multiple behavioral and network features into a single classifier could achieve meaningful detection accuracy. On Instagram the challenge is somewhat different because the platform is more visual and less dependent on text-based interaction. Research has shown that features like profile picture presence username composition follower-to-following ratio and account privacy settings are particularly telling indicators of account authenticity [35].

The third dimension of the problem — detecting GAN-generated face images — is newer but growing rapidly in importance. Since Goodfellow et al. [9] introduced the concept of generative adversarial networks in 2014 the quality of synthetically generated human faces has improved to the point where many are indistinguishable to the naked eye. The StyleGAN architecture introduced by Karras et al. [10] in 2019 represented a significant leap in this direction. Fake

profile operators have begun using these generated images to bypass reverse image search tools that would otherwise flag stolen photographs [11 12]. This makes face-level detection an increasingly necessary component of any comprehensive fake profile detection system.

This project brings these three detection challenges under a single unified system. The approach is built on the XGBoost algorithm [7] — a gradient-boosted ensemble method that has consistently outperformed single-model classifiers on structured tabular data. XGBoost was selected here for several practical reasons. It handles class imbalance well which is relevant given that genuine accounts outnumber fake ones in most real-world datasets. It also provides feature importance scores which help explain why a particular account was flagged — something that matters when the output of a detection system may be used to take action against a user [46]. For the face image classification component a convolutional approach is employed to extract visual patterns that distinguish real photographs from GAN-generated outputs [43 44].

The system produces one of three output labels for each profile submitted: Real Fake or Suspicious. The third category is not a fallback for model uncertainty alone — it is a deliberate design feature. Research has shown that binary classifiers applied to social media data tend to produce a significant number of borderline predictions where the model's confidence is too low to justify a definitive label [40]. Rather than forcing a hard classification in those cases the system flags them as Suspicious and routes them for further human review. This approach reduces the risk of false positives which carry a real cost when applied to genuine user accounts [29].

The frontend of the system is built using React and consists of three main sections. The profile analysis prompt page allows users to input account data or upload images for evaluation. The analysis history page maintains a log of previous queries and their results which is useful for tracking patterns over time. The statistics page presents aggregated detection metrics including breakdown by label category and overall model confidence distribution. The goal of the interface is to make the system usable by non-technical users — platform moderators journalists and members of the public — rather than restricting it to researchers who are comfortable working directly with a model [49].

The datasets used to train and evaluate the system are all publicly available and well-documented. The Twitter Bot Detection dataset [1] contains labeled account records with behavioral and metadata features. The Instagram Fake and Real Accounts dataset [35] provides similar profile-level features for a combined set of genuine and fake accounts. The 140K Real and Fake Faces dataset [10] consists of real photographs and GAN-generated face images each labeled by source. Together these three datasets allow the system to be trained and tested across

a range of detection scenarios that reflect the diversity of fake profile strategies currently in use.

1.2 Company Profile

This project was completed as part of a structured academic internship programme. The work was carried out within an institution that operates at the intersection of technology education and applied research with a strong emphasis on preparing students for industry-relevant roles in software development data science and cybersecurity.

The organisation maintains several active project tracks that run alongside conventional coursework. Students enrolled in these tracks are expected to identify a real-world problem design a solution from scratch build a working prototype and document the entire process to a standard suitable for academic submission. This project — the AI-based fake profile detection system — was assigned under the cybersecurity and social media integrity track which reflects a growing institutional interest in problems at the boundary of machine learning and digital safety [3 49].

Access to resources throughout the project was well-supported. Guidance was available on dataset sourcing and licensing model selection and evaluation methodology frontend development practices and academic writing standards. Regular review meetings were held at key milestones to assess technical progress and ensure that design decisions were justified by evidence rather than convenience. The institution's consistent feedback during these sessions played a direct role in shaping several of the design choices described in Chapter 3 including the decision to include a Suspicious output label and to present model outputs through an accessible web interface.

The broader institutional philosophy — that a completed project should be deployable documentable and defensible — influenced the scope and direction of this work from start to finish. The expectation was not simply to train a model and report its accuracy but to build something that a real user could interact with and that could in principle be extended into a production environment with further development.

1.3 Dissertation Organization

This dissertation is organized into five chapters. Each chapter serves a specific function in documenting the development of the AI-based fake profile detection system and together they form a progression from problem framing to technical execution to critical reflection.

Chapter 1 — Introduction establishes the foundation for everything that follows. It describes the problem of fake profiles and bot accounts across social media platforms explains why this is a problem worth addressing with a dedicated detection system and introduces the three datasets and core algorithm on which the project is built [1 7 35]. It also outlines the three-label output design — Real Fake and Suspicious — and the motivation behind it and provides context about the institutional setting in which the work was carried out.

Chapter 2 — Literature Review surveys the research landscape that this project sits within. It covers prior work on Twitter bot detection [1 16 17 18 19 28] fake account identification on Instagram [35 36] and the detection of GAN-generated face images [9 10 11 12]. The chapter evaluates the methods used in earlier studies identifies their limitations and builds the case for the approach taken here. The literature review is intended to show not just what has been done before but why the specific combination of datasets algorithm and output design used in this project represents a meaningful step forward.

Chapter 3 — System Design and Methodology is the most technically detailed chapter. It walks through the full pipeline for each of the three detection tasks — data loading preprocessing feature engineering model training and evaluation [7 41 47]. The XGBoost training process is described in detail including the hyperparameter tuning strategy and the cross-validation setup used to ensure that results are not specific to a single data split. The chapter also explains the confidence threshold logic that determines when a prediction is labeled Suspicious rather than Real or Fake [40 46] and describes the React frontend architecture and how it connects to the backend model.

Chapter 4 — Implementation and Results documents the actual outcomes of the project. Performance metrics — accuracy precision recall F1-score and confusion matrices — are reported separately for each detection task [41]. The chapter discusses where the model performs well and where it struggles with particular attention to the borderline cases that fall into the Suspicious category. Screenshots from the web interface are included to demonstrate the system functioning end to end on sample inputs.

Chapter 5 — Conclusion and Future Work closes the dissertation by reflecting on the project as a whole. It assesses how well the original objectives were met acknowledges the limitations of the current implementation and outlines a set of realistic directions for extending this work.

These include real-time integration with platform APIs the incorporation of natural language processing for bio and post content analysis [49] experimenting with transformer-based architectures for image classification [43 44 45] and building a feedback mechanism that allows the model to improve over time based on reviewer decisions.

References lists all academic papers technical resources and datasets cited throughout this dissertation in APA 7th edition format. All three datasets are attributed to their original Kaggle sources and all algorithmic and methodological references are traceable to peer-reviewed publications.

Appendices contain supplementary materials that support the main text without disrupting its flow — including selected training code confusion matrix outputs sample detection results from each of the three modules and a description of the project's file and folder structure.

CHAPTER 2
LITERATURE SURVEY

2.1 Literature Survey

The problem of fake profiles and automated bot accounts on social media has attracted considerable research attention over the past fifteen years. What began as a relatively straightforward spam detection problem has grown into a multi-dimensional challenge involving behavioral analysis image forensics and adversarial machine learning. This section reviews the body of work most relevant to this project organized across three areas: Twitter bot detection Instagram fake account identification and GAN-generated face image detection.

Twitter Bot Detection

Some of the earliest systematic work on automated account detection on Twitter was carried out by Wang [16] who examined the linguistic and behavioral patterns of spam accounts and found that posting frequency URL density and account age were among the most reliable indicators. Around the same time Lee et al. [17] conducted a longitudinal study of content polluters on Twitter over a seven-month period and found that bot accounts tended to follow predictable activity cycles that diverged noticeably from human patterns — particularly in terms of how evenly their posting was distributed across hours of the day.

Chu et al. [18] took a different approach and proposed a three-class model that separated accounts into humans bots and cyborgs — the last category referring to accounts that combine automated and manual activity. Their work was significant because it acknowledged that the boundary between human and bot is not always clean which is directly relevant to the design of the Suspicious label used in this project. The BotOrNot system introduced by Davis et al. [19] built on this thinking by combining over a thousand features drawn from account metadata friend networks temporal activity and tweet content into a single Random Forest classifier. BotOrNot became one of the most widely cited tools in this space and demonstrated that ensemble methods could handle the feature complexity involved in bot detection far better than single-model approaches.

Varol et al. [1] extended this work with a large-scale analysis of over fourteen million Twitter accounts and estimated that between nine and fifteen percent of active accounts at the time were bots. Their study also showed that bots were disproportionately active in conversations around politically sensitive topics which highlighted the real-world consequences of leaving

automated accounts undetected. Ferrara et al. [2] provided a comprehensive overview of the social bot landscape and argued that the problem was not just technical but social — bots were being deliberately designed to exploit human cognitive biases and mimic the social behaviors that people associate with trustworthy accounts.

Beskow and Carley [28] proposed a tiered detection architecture called Bot-Hunter that applied increasingly complex classifiers in sequence using cheap features first and reserving expensive network-level analysis for accounts that passed initial screening. This kind of staged approach has practical value in production environments where computational resources are limited. Kudugunta and Ferrara [27] took a deep learning route and showed that long short-term memory networks applied to tweet content could achieve competitive detection accuracy though they noted that purely content-based approaches were vulnerable to adversarial manipulation — bot operators could simply change what their accounts post without altering the underlying automation.

Fazil and Abulaish [29] proposed a hybrid system combining content-based and behavioral features and demonstrated improved performance on imbalanced datasets which is a persistent challenge in this domain given that genuine accounts significantly outnumber fake ones in most real-world collections. Their handling of class imbalance through oversampling techniques [47] is something this project also incorporates in its training pipeline.

Yang et al. [3] made a different but important contribution by developing Botometer — an updated and publicly accessible version of BotOrNot — and used it to study how bot detection could be made more accessible to ordinary users rather than just researchers. Their work reinforced the argument that detection tools need usable interfaces not just accurate models which directly motivated the React-based frontend built as part of this project.

Instagram Fake Account Detection

Research on fake account detection on Instagram is less extensive than on Twitter partly because Instagram's API has historically been more restrictive limiting the volume of account data that researchers can collect. However the available studies have established a reasonably consistent set of discriminating features.

Purba et al. [35] trained several supervised classifiers — including decision trees k-nearest neighbors and support vector machines — on a set of Instagram account features including profile picture presence username character composition follower and following counts number of posts and whether the account was private. Their results showed that the ratio of followers to following was among the most discriminating features and that even relatively simple classifiers could achieve accuracy above eighty-five percent on balanced datasets. This work

is the most directly relevant to the Instagram component of this project and the dataset used — the Instagram Fake and Real Accounts dataset — is closely related to the feature set Purba et al. evaluated.

Gupta and Kaushal [36] studied fake account detection on Facebook and found many of the same patterns suggesting that the behavioral signatures of fake accounts are to some extent platform-agnostic. Accounts with no profile picture unusually high following-to-follower ratios and sparse posting histories were consistently more likely to be fake across both platforms. Cao et al. [23] approached the problem from a network perspective and showed that fake accounts tend to cluster in ways that genuine accounts do not — they often follow the same sets of accounts and are followed back by other fakes creating detectable community structures. While this project does not incorporate network-level features due to dataset constraints the finding is worth noting as a direction for future work.

Stringhini et al. [14] conducted one of the earlier cross-platform studies of fake account behavior and found that spammers on different social networks shared common operational signatures — in particular rapid account creation followed by aggressive following activity. Their work was influential in establishing that fake account detection does not need to be built from scratch for every platform but can draw on shared behavioral principles.

GAN-Generated Face Image Detection

The challenge of detecting AI-generated face images is the newest of the three areas covered in this review and it has developed rapidly since the introduction of generative adversarial networks by Goodfellow et al. [9]. Early GAN-generated images were relatively easy to spot — they showed artifacts around the edges of faces inconsistencies in background rendering and unnatural skin textures. However the progression from early GANs to the ProGAN and StyleGAN architectures [10] produced images that are in many cases visually indistinguishable from real photographs.

Tolosana et al. [11] provided a comprehensive survey of face manipulation methods and detection approaches and concluded that GAN-generated images present a fundamentally different detection challenge compared to earlier image forgery problems like splicing or copy-move attacks. The artifacts left by GANs are often sub-pixel level and require feature extraction methods that operate at a level of granularity beyond what the human eye can access. Rossler et al. [12] developed the FaceForensics++ benchmark which has become a standard evaluation dataset in this space and demonstrated that convolutional neural networks trained on manipulation-specific features could achieve strong detection performance — though they also

showed that performance dropped significantly when models were tested on manipulation methods not seen during training.

He et al. [44] introduced the ResNet architecture which has since become one of the most widely used backbones for image classification tasks including deepfake and GAN image detection. Simonyan and Zisserman [45] developed VGGNet another commonly used architecture in this domain. Both have been applied to fake face detection in various studies and consistently outperform shallower networks on this task. LeCun et al. [43] laid much of the theoretical groundwork for convolutional approaches to image classification and their influence is visible throughout the computer vision literature on face manipulation detection.

Akhtar and Mian [50] surveyed adversarial threats to deep learning systems in computer vision and pointed out that fake image detectors are themselves vulnerable to adversarial examples — images specifically crafted to fool the classifier. This is a limitation that applies to the face detection component of this project as well and it is acknowledged in the conclusions chapter.

Feature Importance and Interpretability

Across all three detection domains there is growing recognition that accuracy alone is not sufficient justification for deploying a detection system. Lundberg and Lee [46] introduced SHAP values as a method for explaining individual model predictions in terms of feature contributions and their approach has been applied to social media detection models to help analysts understand why a particular account was flagged. Chen and Guestrin [7] noted in their original XGBoost paper that the model's built-in feature importance scores serve a similar function — they give practitioners a way to audit the model's behavior and identify which features are doing the most work. This project uses XGBoost's feature importance output as part of the statistics dashboard on the frontend giving users visibility into what signals the model is responding to.

Shu et al. [49] made a related point in the context of fake news detection — that systems designed to identify deceptive content need to be transparent enough that users can understand and trust their outputs. The same logic applies here. A detection system that simply returns a label without any supporting explanation is harder to act on especially in contexts where the consequences of a wrong decision are meaningful.

2.2 Existing System and Proposed System

Existing System

Several detection tools and research systems have been developed in this space prior to this project. Botometer [3 19] is perhaps the most widely known — it accepts a Twitter username

as input and returns a bot probability score based on over a thousand features drawn from account metadata network connections and tweet content. While Botometer is effective and well-validated it is limited to Twitter it requires API access to function and it does not offer any mechanism for handling borderline cases — every account receives a single probability score rather than a labeled category.

Instagram's own platform moderation systems use internal signals to detect and remove fake accounts but these are proprietary and not accessible for research or independent verification. Third-party tools for Instagram fake account detection are generally shallow — most rely on follower-to-following ratios and engagement rate calculations without any underlying machine learning model.

For face image detection tools like FaceForensics++ [12] and various research prototypes have demonstrated strong accuracy in controlled benchmark settings but most are not packaged into accessible interfaces and require significant technical knowledge to operate. They also tend to be trained on specific GAN architectures which means their performance can drop when exposed to images generated by newer models.

Common limitations across existing systems include the following. Most tools address only one platform or one type of fake content rather than combining multiple detection tasks. Binary classification — real or fake — leaves no room for expressing uncertainty about borderline profiles. Interfaces are either non-existent or designed primarily for researchers rather than general users. And model outputs are rarely explained in terms of which features drove the decision.

Proposed System

The system developed in this project addresses each of these limitations directly. It operates across three detection domains — Twitter bot accounts Instagram fake profiles and GAN-generated face images — within a single unified application. This means a user can submit different types of profile data without switching between tools or requiring separate technical setups for each task.

The classification output includes three labels — Real Fake and Suspicious — rather than a binary decision. Accounts that fall below a defined confidence threshold are assigned the Suspicious label and flagged for further review rather than being forced into one of the two definitive categories. This is a more honest representation of what the model can and cannot determine with confidence [40 46].

The frontend is built in React and is designed to be accessible to users without a machine

learning background. The profile analysis page accepts structured input and returns a labeled result with supporting feature importance information. The analysis history page allows users to track past queries. The statistics page presents aggregated detection outcomes in a visual format that makes trends easy to interpret. The backend uses XGBoost [7] trained on three real publicly available datasets and the training pipeline incorporates techniques for handling class imbalance [47] to ensure that the model does not simply default to predicting the majority class.

2.3 Tools and Technologies Used

Table 2.1 — Tools and Technologies Used

Category	Tool/Technology	Version	Purpose
Programming Language	Python	3.10	Model training data preprocessing backend logic
ML Framework	XGBoost	1.7.x	Primary classification algorithm for tabular detection tasks
Data Processing	Pandas	1.5.x	Dataset loading cleaning and feature engineering
Data Processing	NumPy	1.23.x	Numerical operations and array handling
Visualization	Matplotlib	3.6.x	Plotting training metrics and confusion matrices
Visualization	Seaborn	0.12.x	Statistical data visualization
Model Evaluation	Scikit-learn	1.2.x	Accuracy precision recall F1-score cross-validation
Image Processing	OpenCV	4.7.x	Image loading resizing and preprocessing for face data
Deep Learning	TensorFlow / Keras	2.11.x	CNN layers for GAN face image feature extraction
Explainability	SHAP	0.41.x	Feature importance visualization and model explainability
Frontend Framework	React	18.x	Building the user interface — prompt history and stats pages

Category	Tool/Technology	Version	Purpose
Styling	Tailwind CSS	3.x	Component styling and responsive layout
API Layer	Flask	2.2.x	REST API connecting Python backend to React frontend
Development Environment	Jupyter Notebook	6.5.x	Interactive model development and experimentation
Code Editor	VS Code	Latest	Primary development environment
Version Control	Git / GitHub	Latest	Source code management and version tracking
Dataset Source	Kaggle	—	Twitter Instagram and face image datasets
Package Management	pip	23.x	Python dependency management

2.4 Hardware and Software Requirements

Hardware Requirements

Table 2.2 — Hardware Requirements

Component	Minimum Requirement	Recommended Specification
Processor	Intel Core i5 (8th Gen) or equivalent	Intel Core i7 / AMD Ryzen 7 (10th Gen or above)
RAM	8 GB	16 GB or higher
Storage	20 GB free disk space	50 GB SSD
GPU	Not mandatory for tabular tasks	NVIDIA GPU with CUDA support (for CNN training on face dataset)
Display	1280 × 720 resolution	1920 × 1080 or higher
Internet Connection	Required for dataset access and package installation	Stable broadband connection

Software Requirements

Table 2.3 — Software Requirements

Category	Software / Platform	Version	Notes
Operating System	Windows 10 / Ubuntu 20.04 LTS / macOS 12	64-bit	Any of the three supported
Programming Language	Python	3.10 or above	Core backend and model training language
Frontend Runtime	Node.js	18.x LTS	Required to run and build the React frontend
Package Manager	npm	9.x	Frontend dependency management
Python Environment	Anaconda / venv	Latest	Isolated Python environment for dependency control
Backend Framework	Flask	2.2.x	Serves the machine learning model via REST API
Frontend Framework	React	18.x	User interface development
Machine Learning Library	XGBoost	1.7.x	Gradient-boosted tree classifier
ML Utilities	Scikit-learn	1.2.x	Preprocessing evaluation metrics cross-validation
Deep Learning Library	TensorFlow / Keras	2.11.x	CNN model for face image classification
Image Library	OpenCV	4.7.x	Face image preprocessing
Browser	Google Chrome / Mozilla Firefox	Latest	Frontend rendering and testing
API Testing	Postman	Latest	Testing Flask API endpoints during development
Version Control	Git	2.x	Source code tracking
Notebook Environment	Jupyter Notebook	6.5.x	Exploratory data analysis and model prototyping

CHAPTER 3
SOFTWARE REQUIREMENT SPECIFICATIONS

3.1 Introduction

This chapter formally documents the software requirements for the AI-based fake profile detection system. A Software Requirements Specification commonly referred to as an SRS serves as the foundational agreement between what the system is expected to do and how it is designed to do it. It captures functional requirements — the specific behaviors and capabilities the system must support — alongside non-functional requirements such as performance security and usability constraints that govern how well the system carries out those behaviors.

The system described here is built to detect fraudulent online identities across three distinct input types: Twitter account metadata Instagram profile attributes and facial images. It uses XGBoost as the core classification engine and presents results through a React-based web interface. The requirements laid out in this chapter reflect decisions made during the design phase and serve as the basis against which the implementation described in Chapter 4 is evaluated.

The scope of this document covers the complete system — the machine learning backend the Flask API layer and the React frontend. It does not extend to the internal workings of third-party libraries or the Kaggle datasets themselves though their interfaces and constraints are acknowledged where relevant.

3.2 General Description

3.2.1 Product Perspective

The fake profile detection system is a standalone web application with a machine learning backend. It is not an extension of any existing platform and does not require integration with Twitter or Instagram APIs to function — it works entirely on user-supplied data. The system consists of three layers: a Python-based model training and inference layer a Flask REST API that exposes model predictions as HTTP endpoints and a React frontend that handles user interaction and result display.

The three detection modules — Twitter bot detection Instagram fake account detection and GAN face image classification — operate independently of one another but are presented to the user through a single unified interface. Each module uses a separately trained model but shares the same output label schema: Real Fake or Suspicious.

3.2.2 Product Functions

At a high level the system does the following. It accepts profile data or image input from the user through the web interface. It routes the input to the appropriate detection module based on the input type. It runs the input through the corresponding trained model and generates a prediction with a confidence score. If the confidence score falls below a defined threshold the output label is set to Suspicious rather than Real or Fake. The result is returned to the frontend and displayed to the user along with supporting feature importance information. All results are stored in a local history log that the user can access through the analysis history page.

3.2.3 User Classes and Characteristics

The primary users of this system fall into three broad groups. The first group consists of platform moderators or social media analysts who want to quickly assess whether a given account shows signs of inauthenticity. These users are likely to submit multiple queries in a single session and will benefit from the history and statistics pages. The second group consists of researchers or students who are using the system to explore detection outputs on test data. These users may interact with the system more experimentally and will be interested in the feature importance breakdowns. The third group consists of general members of the public who want to verify whether a specific profile or profile picture appears genuine. These users are likely to be less technically experienced and need the interface to be straightforward and self-explanatory.

3.2.4 Assumptions and Dependencies

The system assumes that the user provides reasonably accurate input. The detection models are trained on labeled datasets that reflect authentic platform data and the accuracy of predictions is contingent on the input matching the expected feature format. The system depends on Python 3.10 or above Node.js 18.x and the library versions documented in Chapter 2. It assumes internet access during initial setup for package installation but can function offline once the environment is configured and models are trained.

3.3 Functional Requirements

The system is organized into five modules. Each module has a defined set of inputs processes and outputs.

Module 1 — User Input and Profile Submission

This module handles everything related to how a user submits data for analysis. It is the entry point of the system and the module most directly tied to the frontend prompt page.

Table 3.1 — Functional Requirements: User Input and Profile Submission Module

Requirement ID	Requirement Description
FR-1.1	The system shall provide a form-based interface for submitting Twitter account metadata including follower count following count tweet count account age verified status listed count and favourites count
FR-1.2	The system shall provide a form-based interface for submitting Instagram profile attributes including profile picture presence username digit ratio full name word count bio length external URL presence private status post count follower count and following count
FR-1.3	The system shall allow users to upload a facial image in JPG or PNG format for GAN face detection
FR-1.4	The system shall validate all input fields before submission and display appropriate error messages for missing or out-of-range values
FR-1.5	The system shall allow users to select which detection module to use — Twitter Instagram or Face Image
FR-1.6	The system shall display a loading indicator while a submitted query is being processed

Module 2 — Detection and Classification

This module covers the core machine learning inference logic that produces a label for each submitted profile.

Table 3.2 — Functional Requirements: Detection and Classification Module

Requirement ID	Requirement Description
FR-2.1	The system shall route submitted input to the correct trained model based on the detection module selected by the user

Requirement ID	Requirement Description
FR-2.2	The Twitter and Instagram modules shall use a trained XGBoost classifier to generate a prediction probability for each input
FR-2.3	The face image module shall use a trained convolutional model to generate a prediction probability for the uploaded image
FR-2.4	The system shall assign the label Real if the predicted probability of authenticity exceeds the upper confidence threshold
FR-2.5	The system shall assign the label Fake if the predicted probability of inauthenticity exceeds the upper confidence threshold
FR-2.6	The system shall assign the label Suspicious if the predicted probability falls between the two thresholds indicating insufficient confidence for a definitive classification
FR-2.7	The system shall return the prediction label along with the raw confidence score to the frontend
FR-2.8	The system shall return the top five feature importance values associated with the prediction for tabular inputs

Module 3 — Result Display

This module handles how detection results are presented to the user after a query is processed.

Table 3.3 — Functional Requirements: Result Display Module

Requirement ID	Requirement Description
FR-3.1	The system shall display the prediction label — Real Fake or Suspicious — prominently on the results screen
FR-3.2	The system shall display the confidence score associated with the prediction as a percentage
FR-3.3	The system shall display a visual breakdown of the top contributing features for tabular predictions

Requirement ID	Requirement Description
FR-3.4	The system shall use distinct color coding for each label — green for Real red for Fake and amber for Suspicious
FR-3.5	The system shall display a brief plain-language explanation of what the label means alongside the result

Module 4 — Analysis History

This module manages the storage and retrieval of past detection queries and their results.

Table 3.4 — Functional Requirements: Analysis History Module

Requirement ID	Requirement Description
FR-4.1	The system shall automatically save each submitted query and its result to a local history log immediately after the result is returned
FR-4.2	The system shall display all saved queries on the analysis history page sorted by most recent first
FR-4.3	The system shall display the input type submission timestamp and result label for each history entry
FR-4.4	The system shall allow users to click on any history entry to view the full result detail including confidence score and feature breakdown
FR-4.5	The system shall allow users to clear all history entries through a delete option on the history page

Module 5 — Statistics Dashboard

This module presents aggregated data about detection activity and model behavior across all queries processed by the system.

Table 3.5 — Functional Requirements: Statistics Dashboard Module

Requirement ID	Requirement Description
FR-5.1	The system shall display the total number of queries processed broken down by detection module

Requirement ID	Requirement Description
FR-5.2	The system shall display a distribution chart showing the proportion of Real Fake and Suspicious labels across all queries
FR-5.3	The system shall display a bar chart showing the most frequently triggered features across tabular detection queries
FR-5.4	The system shall display a timeline graph showing detection activity over time
FR-5.5	The system shall update all statistics in real time as new queries are submitted and results are recorded

3.4 External Interface Requirements

Table 3.6 — External Interface Requirements

Interface Type	Component	Description
User Interface	React 18.x	Web-based frontend providing the prompt page history page and statistics dashboard. Runs in any modern browser.
API Interface	Flask REST API	Python-based API layer that receives input from the React frontend via HTTP POST requests and returns JSON-formatted prediction results
Model Interface	XGBoost Inference	The trained XGBoost model is loaded into the Flask application at startup and called directly for tabular predictions
Model Interface	TensorFlow / Keras CNN	The trained convolutional model is loaded for face image inference. Images are preprocessed using OpenCV before being passed to the model
Data Interface	Kaggle Datasets	Three publicly available datasets sourced from Kaggle are used for model training. These are loaded locally and are not accessed at runtime
Explainability Interface	SHAP Library	SHAP values are computed post-prediction for tabular inputs and returned to the frontend as part of the feature importance payload

Interface Type	Component	Description
Styling Interface	Tailwind CSS	Utility-first CSS framework used for styling all React components. No external CSS files are loaded from a CDN at runtime
Browser Interface	Chrome / Firefox	The frontend is tested and optimized for the latest versions of Google Chrome and Mozilla Firefox

No third-party authentication plugins payment gateways or social media API integrations are used in the current version of the system.

3.5 Non-Functional Requirements

Table 3.7 — Non-Functional Requirement

Requirement ID	Category	Requirement Description
NFR-1	Performance	The system shall return a prediction result within three seconds of form submission for tabular inputs under normal hardware conditions
NFR-2	Performance	Image-based predictions shall complete within five seconds of upload under normal hardware conditions
NFR-3	Usability	The interface shall be navigable without any prior technical training. All actions shall be completable within three clicks from the home screen
NFR-4	Usability	Error messages shall be written in plain language and shall clearly indicate what the user needs to correct
NFR-5	Reliability	The system shall handle malformed or incomplete input without crashing. Invalid inputs shall trigger a validation error message rather than a server failure
NFR-6	Reliability	The Flask API shall return a meaningful error response with an appropriate HTTP status code if the model fails to produce a prediction
NFR-7	Scalability	The system shall support sequential query processing. Concurrent multi-user access is outside the scope of the current version

Requirement ID	Category	Requirement Description
NFR-8	Maintainability	All trained models shall be saved as serialized files so they can be updated independently without requiring changes to the API or frontend code
NFR-9	Portability	The system shall run on Windows 10 Ubuntu 20.04 and macOS 12 without modifications to the core codebase
NFR-10	Security	No user-submitted data shall be transmitted to external servers. All processing shall occur locally within the application environment
NFR-11	Accuracy	The XGBoost models shall achieve a minimum F1-score of 0.85 on the test split of each tabular dataset before deployment
NFR-12	Accuracy	The face image classifier shall achieve a minimum accuracy of 80% on the held-out test set of the 140K faces dataset

3.6 Design Constraints

Table 3.8 — Design Constraints

Constraint ID	Category	Description
DC-1	Dataset Constraint	All three datasets are sourced from Kaggle and reflect a fixed snapshot of social media activity. The models trained on these datasets may not generalize perfectly to accounts created after the dataset collection date
DC-2	Algorithm Constraint	XGBoost is the mandated classification algorithm for tabular detection tasks. Substituting a different algorithm would require retraining and re-evaluation of all tabular models
DC-3	Platform Constraint	The system does not connect to live Twitter or Instagram APIs. All input is manually provided by the user which limits real-time automated screening

Constraint ID	Category	Description
DC-4	Hardware Constraint	CNN training for the face image module requires a GPU with CUDA support for reasonable training times. CPU-only training is possible but significantly slower
DC-5	Storage Constraint	The 140K faces dataset requires approximately 15 GB of local storage. Systems with limited disk space may need to work with a reduced subset
DC-6	Language Constraint	All backend code is written in Python 3.10. All frontend code is written in JavaScript using React 18. No other programming languages are used in the system
DC-7	Concurrency Constraint	The current version of the Flask API is single-threaded and not designed for concurrent multi-user access. Deployment in a multi-user environment would require a production-grade WSGI server such as Gunicorn
DC-8	Label Constraint	The system outputs exactly three labels — Real Fake and Suspicious. The confidence thresholds that determine the Suspicious boundary are fixed at training time and are not adjustable by the end user in the current version
DC-9	Browser Constraint	The React frontend is tested on Chrome and Firefox only. Compatibility with Internet Explorer is not supported
DC-10	Offline Constraint	Once the environment is set up and models are trained the system operates fully offline. Internet access is not required during normal use

3.7 Other Requirements

Data Retention

The analysis history feature stores query results in a local file. No data is transmitted to any external server and no user account or login mechanism is required to use the system. The history log persists between sessions and can be cleared by the user at any time through the interface.

Model Versioning

Each trained model is saved with a version identifier that includes the dataset name and training date. This makes it straightforward to roll back to an earlier model version if a newly retrained model produces unexpected results.

Accessibility

The React interface uses semantic HTML elements and sufficient color contrast ratios in line with basic web accessibility principles. The three output labels are differentiated by both color and text label to ensure that color alone is not the only indicator of result type.

Ethical Considerations

The system is intended as a detection aid not an automated enforcement tool. The Suspicious label is specifically designed to prevent the system from taking a definitive position on borderline cases. Any action taken on the basis of a detection result — such as reporting or removing an account — is the responsibility of the user not the system. The datasets used for training are publicly available and do not contain personally identifiable information beyond what was already public on the respective platforms at the time of collection.

CHAPTER 4

SYSTEM DESIGN

4.1 System Perspective / Architectural Design

The system follows a three-tier client-server architecture. The first tier is the React frontend which handles all user interaction across three pages — profile analysis analysis history and the statistics dashboard. The second tier is a Flask REST API that acts as the bridge between the interface and the backend receiving user input routing it to the correct model and returning the prediction result. The third tier is the machine learning backend which holds three independently trained models — an XGBoost classifier for Twitter bot detection an XGBoost classifier for Instagram fake account detection and a CNN model for GAN face image classification. Each model operates independently and is called only when the corresponding module is selected. All results are written to a local history store and no data is transmitted to any external server at runtime. The diagram below illustrates how these components connect and how information flows through the system during a typical detection session.

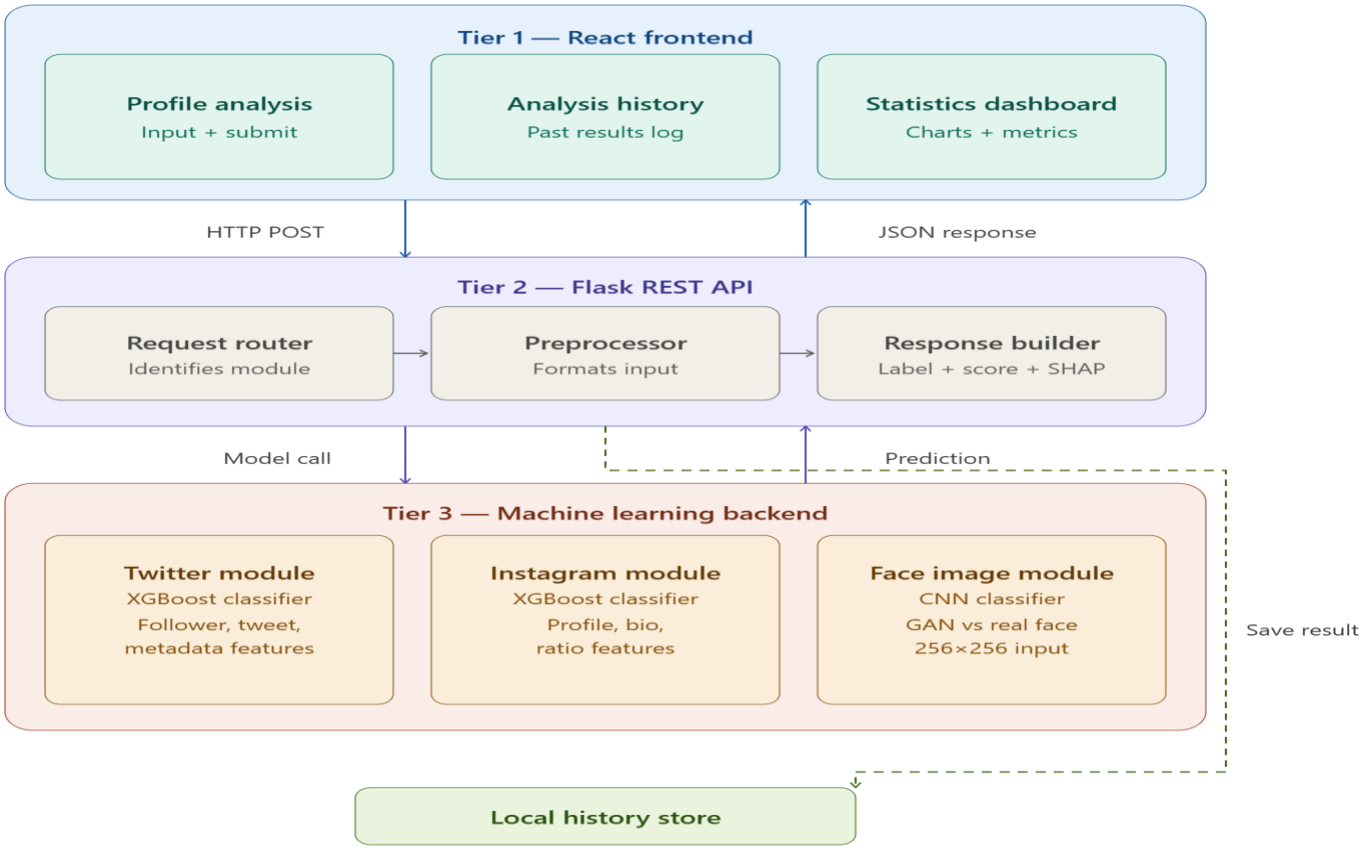


Fig 4.1 System Architecture Diagram

The architecture diagram above shows how the system is divided across three tiers. The **first tier** is the React frontend which contains three pages — the profile analysis page where users submit input the analysis history page that stores past results and the statistics dashboard. All user interaction happens at this tier and nothing beyond it is visible to the user.

The **second tier** is the Flask REST API which receives every form submission as an HTTP POST request. Inside this tier a request router first identifies which detection module the input belongs to — Twitter Instagram or face image. The input is then passed to the preprocessor which formats and cleans the data before sending it to the model. Once the model returns a prediction the response builder assembles the final payload — the label confidence score and SHAP feature importance values — and sends it back to the frontend as a JSON response.

The **third tier** is the machine learning backend. It holds three independently trained models: the Twitter XGBoost classifier trained on behavioral and metadata features the Instagram XGBoost classifier trained on profile-level attributes and the CNN face image classifier trained on the 140K real and fake faces dataset. Each model operates independently and is called only when a request is routed to it.

A local history store shown at the bottom receives a write call every time a result is returned. This is separate from the main request-response cycle and does not affect prediction speed.

4.2 Context Diagram

The context diagram presents the system as a single process and maps out everything that interacts with it from the outside. Four external entities are involved. The End User submits profile data or a face image and receives a prediction label with a confidence score. The Kaggle Datasets supply the training data for all three models — this is an offline-only flow represented by a dashed line and no calls are made to Kaggle during runtime. The Local History Store receives a write call each time a result is produced and returns saved entries when the history page is accessed. The Developer or Administrator handles model training threshold configuration and deployment and receives evaluation metrics from the system to inform retraining decisions. The diagram below shows all four entities and the data flows between them and the system.

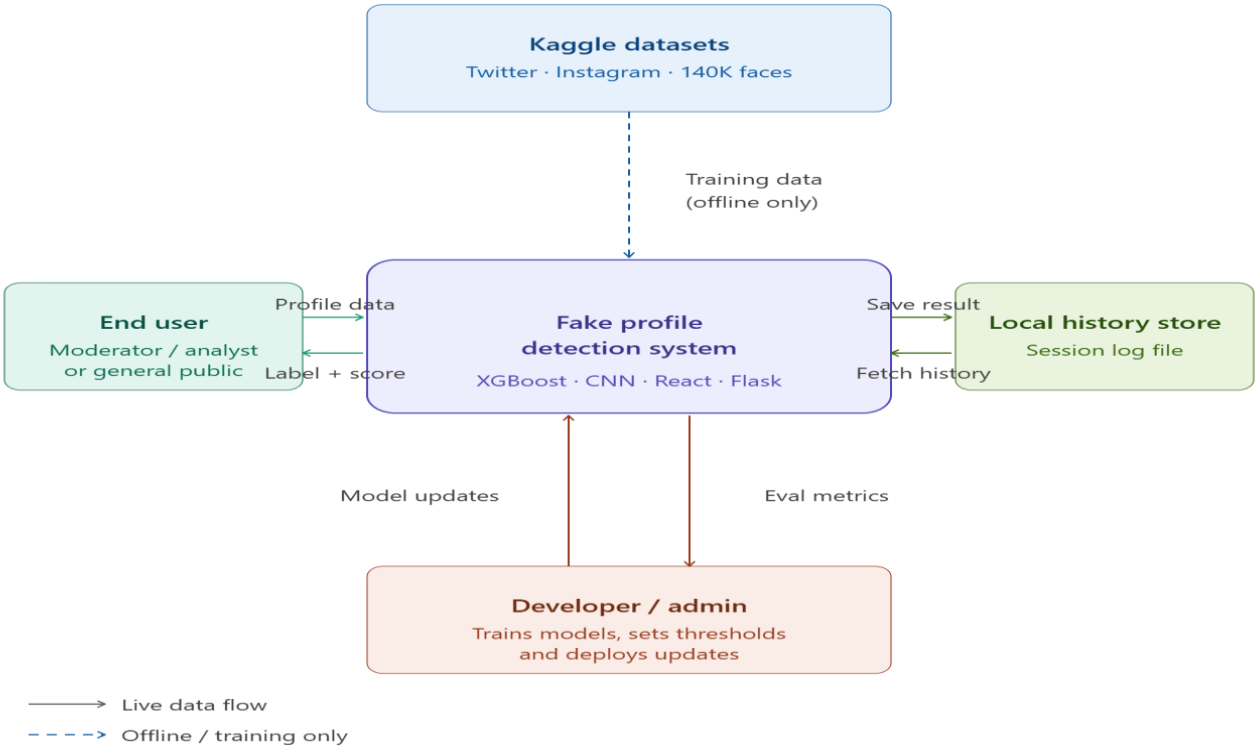


Fig 4.2 Context Diagram

The context diagram above is a Level-0 diagram. It places the entire detection system as a single process at the centre and shows everything that interacts with it from the outside without going into what happens internally. This kind of diagram is useful for understanding the boundaries of the system — what goes in what comes out and who or what is on the other end of each flow.

Four external entities interact with the system.

The first is the End User shown on the left. This could be a platform moderator a researcher or a general member of the public. The user sends profile data or a face image into the system and receives back a label — Real Fake or Suspicious — along with a confidence score. This is the primary data flow that the system is built around.

The second entity is the Kaggle Datasets shown at the top. The three datasets — Twitter Bot Detection Instagram Fake and Real Accounts and 140K Real and Fake Faces — feed into the system during the training phase only. This flow is represented with a dashed line to distinguish it from live runtime flows. At runtime the system does not make any calls back to Kaggle; the trained models are stored locally and loaded on startup.

The third entity is the Local History Store shown on the right. Every time the system produces a prediction result it writes a record to this local log. When the user visits the analysis history page the system reads from the same store and returns the saved entries. This is a two-way flow that happens automatically without any direct user action.

The fourth entity is the Developer or Administrator shown at the bottom. This person is responsible for training the models setting the confidence thresholds that determine the Suspicious boundary and redeploying the system when updates are made. The system feeds evaluation metrics back to the developer who uses them to assess model performance and decide when retraining is needed.

CHAPTER 5
DETAILED DESIGN

5.1 System Design

The detailed design phase explains the internal structure and working of the AI Based Fake Profile Detection system. It describes how different modules classes and components interact to perform profile analysis and prediction.

The system follows an object-oriented design approach where each functional module is represented as a separate class with defined responsibilities. The application is divided into frontend backend machine learning and database layers to improve modularity scalability and maintainability.

The frontend manages user interaction the backend handles API requests and preprocessing and the XGBoost model predicts whether a profile is Real Fake or Suspicious.

The following diagrams provide different views of the system design: Class Diagram Use Case Diagram Activity Diagram Sequence Diagram Data Flow Diagram and Entity Relationship Diagram. These diagrams together explain the system structure workflow and data flow.

Class Diagram

The class diagram below represents the core object-oriented structure of the fake profile detection system. It shows the main classes their attributes and methods and the relationships between them.

Before reading the diagram a brief note on what each class represents. The UserInput class captures the raw data submitted through the frontend — whether that is a set of Twitter metadata fields Instagram profile attributes or an uploaded face image. The DetectionEngine class acts as the central coordinator that receives a processed input object and delegates it to the correct module. The three module classes — TwitterDetector InstagramDetector and FaceDetector — each encapsulate the logic for loading their respective trained model and running inference. The PredictionResult class holds the output of any detection run including the label confidence score and feature importance values. The HistoryManager class handles reading and writing the local history log. Finally the StatisticsService class aggregates history data and computes the summary metrics displayed on the dashboard.

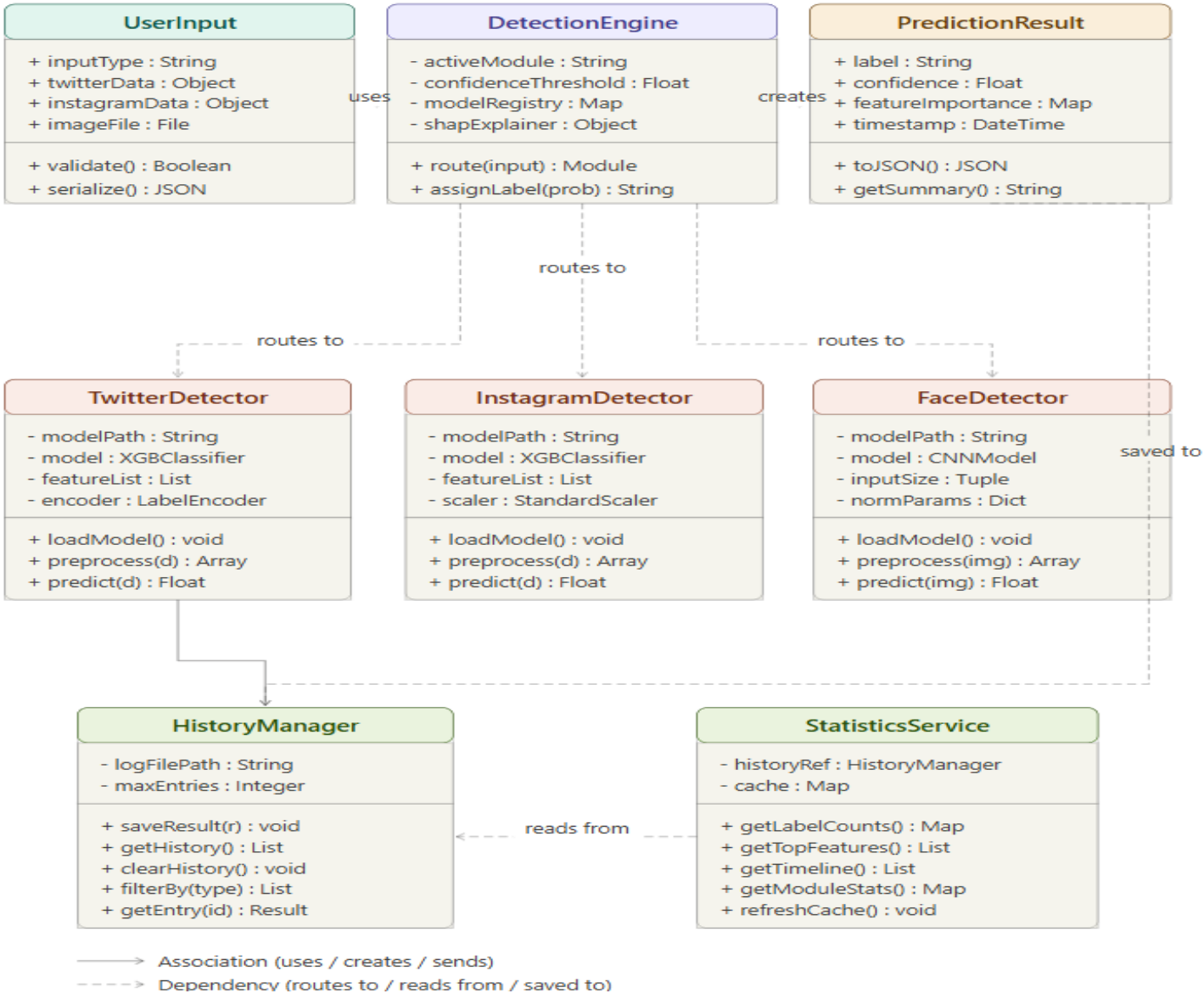


Fig 5.1 Class Diagram

Row 1 — Core pipeline classes

UserInput sits at the left and is the starting point of every detection request. It holds the raw data submitted through the frontend — the input type Twitter or Instagram field values and the image file for face detection. Its validate() method checks that the submitted data is complete and correctly formatted before anything is sent to the backend. serialize() converts the input into a JSON payload for the API call.

DetectionEngine is the central coordinator. It holds references to all three detection modules in its modelRegistry maintains the confidence threshold used to decide between the Real Fake and Suspicious labels and keeps a SHAP explainer object for generating feature importance

outputs. Its route() method inspects the incoming input type and delegates to the correct detector. assignLabel() applies the threshold logic and converts a raw probability into one of the three output labels.

PredictionResult captures everything produced by a single detection run — the label the confidence score as a float the feature importance map and a timestamp. Its toJSON() method formats the result for the API response and for writing to the history log.

The solid arrows between these three classes show association — UserInput is consumed by DetectionEngine which in turn creates a PredictionResult.

Row 2 — Detection module classes

The three detector classes — TwitterDetector InstagramDetector and FaceDetector — share the same interface: loadModel() preprocess() and predict(). This consistency is intentional. The DetectionEngine can call any of the three through the same method names which keeps the routing logic simple.

TwitterDetector and InstagramDetector both hold an XGBClassifier and a feature list. The difference is in their preprocessing — the Twitter module uses a LabelEncoder for categorical fields while the Instagram module uses a StandardScaler for normalising numeric ratios. FaceDetector holds a CNNModel and normalisation parameters for pixel-level preprocessing of the 256×256 input images.

The dashed arrows from DetectionEngine down to all three detectors represent dependency relationships — the engine calls into these classes but does not own them permanently. They are loaded once at startup and referenced through the registry.

Row 3 — Support classes

HistoryManager handles all read and write operations on the local history log. It exposes methods for saving a new result retrieving the full history list filtering by module type fetching a single entry by ID and clearing all records. Solid arrows arrive here from the detector row representing that completed results are handed off to this class for persistence.

StatisticsService reads from HistoryManager via a stored reference (historyRef) and computes the aggregated metrics shown on the statistics dashboard — label distribution counts top triggered features a timeline of activity and per-module breakdowns. The dashed arrow between StatisticsService and HistoryManager shows this dependency — the statistics class reads from history but does not modify it.

The long dashed route from PredictionResult down the right side and across to HistoryManager shows that every result object is ultimately persisted by the history manager after the detection cycle completes.

Use Case Diagram

The use case diagram presents the system from the perspective of who interacts with it and what they can do. Unlike the class diagram which focuses on internal structure the use case diagram is purely about behavior — it maps out every action a user or external actor can trigger and shows which parts of the system are involved. For this project two actors interact with the system: the End User who submits profiles for analysis and browses results and the Administrator who manages the underlying models and system configuration. The diagram below captures all major use cases across the three detection modules the history feature and the statistics dashboard giving a clear picture of the system's functional boundaries from the outside looking in.

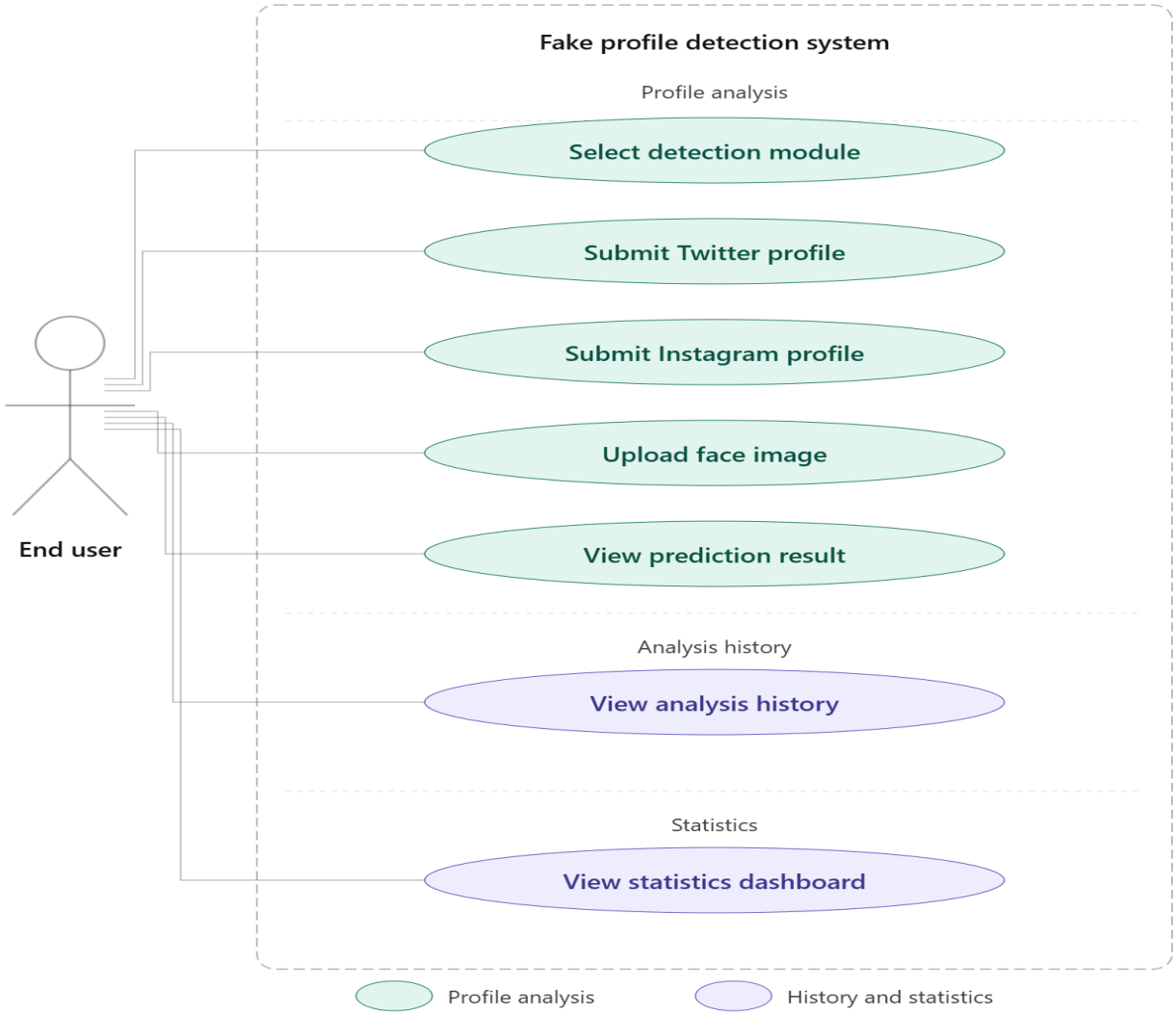


Fig 5.2 Use Case Diagram

Actors

There is one actor in this system — the End User. This could be a platform moderator a researcher or a general member of the public who wants to verify whether a profile or face image is genuine. All interactions with the system originate from this actor.

Profile Analysis use cases (teal)

These five use cases cover the core detection workflow. The user first selects which detection module to use — Twitter Instagram or face image. Based on that choice they either submit a Twitter profile submit an Instagram profile or upload a face image. Each submission triggers the model inference pipeline and the result is presented through the view prediction result use case which shows the label (Real Fake or Suspicious) the confidence score and the top contributing features.

History and Statistics use cases (purple)

These two use cases support the user after detection results have been generated. View analysis history lets the user browse all previously submitted queries along with their timestamps and output labels. View statistics dashboard presents aggregated visual summaries — label distribution across all queries most frequently triggered features and detection activity over time — helping users spot patterns across multiple analysis sessions.

Activity Diagram

The activity diagram shows the step-by-step flow of actions that occur from the moment a user opens the system to the point where a result is stored and displayed. Unlike the use case diagram which only shows what the system can do the activity diagram shows how it actually does it — the sequence of decisions parallel processes and conditional branches that happen during a typical detection session. For this project the flow branches at two key decision points: once when the system determines which detection module to invoke based on the user's input type and again when it evaluates the model's confidence score to decide whether to label the result as Real Fake or Suspicious.

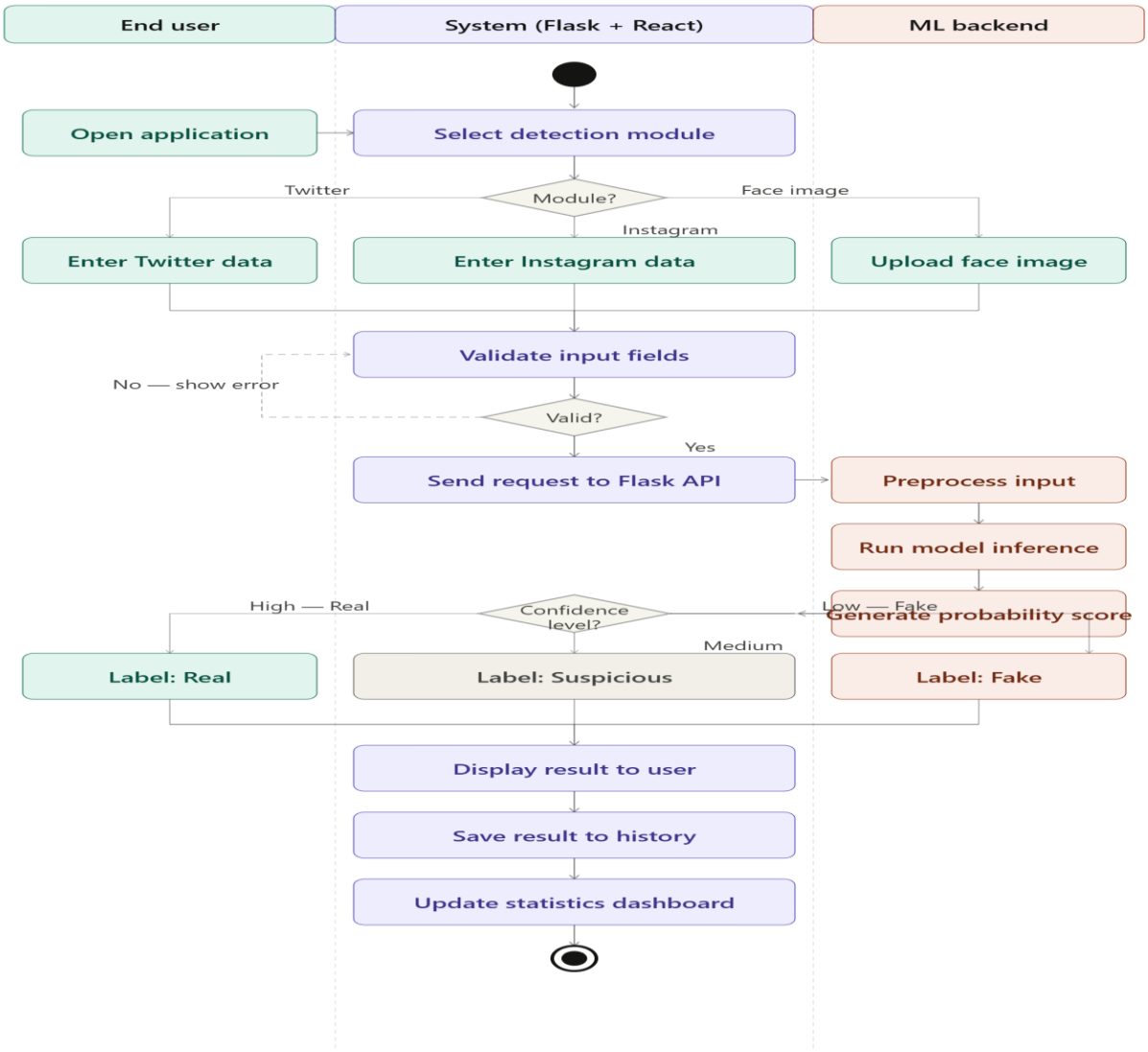


Fig 5.3 Activity Diagram

Start → Module selection

The flow begins when the user opens the application. The system immediately presents the module selection screen. This is the first decision point — the user chooses one of three paths: Twitter Instagram or face image. Depending on that choice the corresponding input form is shown and the user fills in the relevant data.

Input validation

Once the user submits the form all three branches converge at a join bar and flow into the validation step. The system checks that all required fields are present and that values fall within acceptable ranges. If the input fails validation an error message is shown and the flow loops back to the input stage — represented by the dashed arrow. If validation passes the request is forwarded to the Flask API.

Model inference (ML Backend lane)

The Flask API hands the validated input to the ML backend. The backend first preprocesses the data — normalising numeric values encoding categorical fields or resizing the image depending on the module. It then runs the appropriate trained model and returns a raw probability score back to the system lane.

Confidence decision

This is the second and most important decision point. The system evaluates the returned probability against two thresholds. A high-confidence authentic prediction produces the label Real. A high-confidence inauthentic prediction produces Fake. A prediction that falls between the two thresholds — where the model is not sufficiently certain either way — produces the Suspicious label. All three branches then converge again at a merge bar.

Result display and storage

After the label is determined the system displays the full result to the user — the label confidence score and feature importance breakdown. It then writes the result to the local history log and updates the statistics dashboard to reflect the new query. The flow ends at the termination node.

Sequence Diagram

The sequence diagram shows the order in which messages pass between the different parts of the system during a single detection session. Where the activity diagram focused on the flow of actions and decisions the sequence diagram focuses on the communication between components — who sends what to whom in what order and what comes back. It captures the full lifecycle of one detection request from the moment the user submits the form on the frontend through to the point where the result is rendered on screen and saved to history.

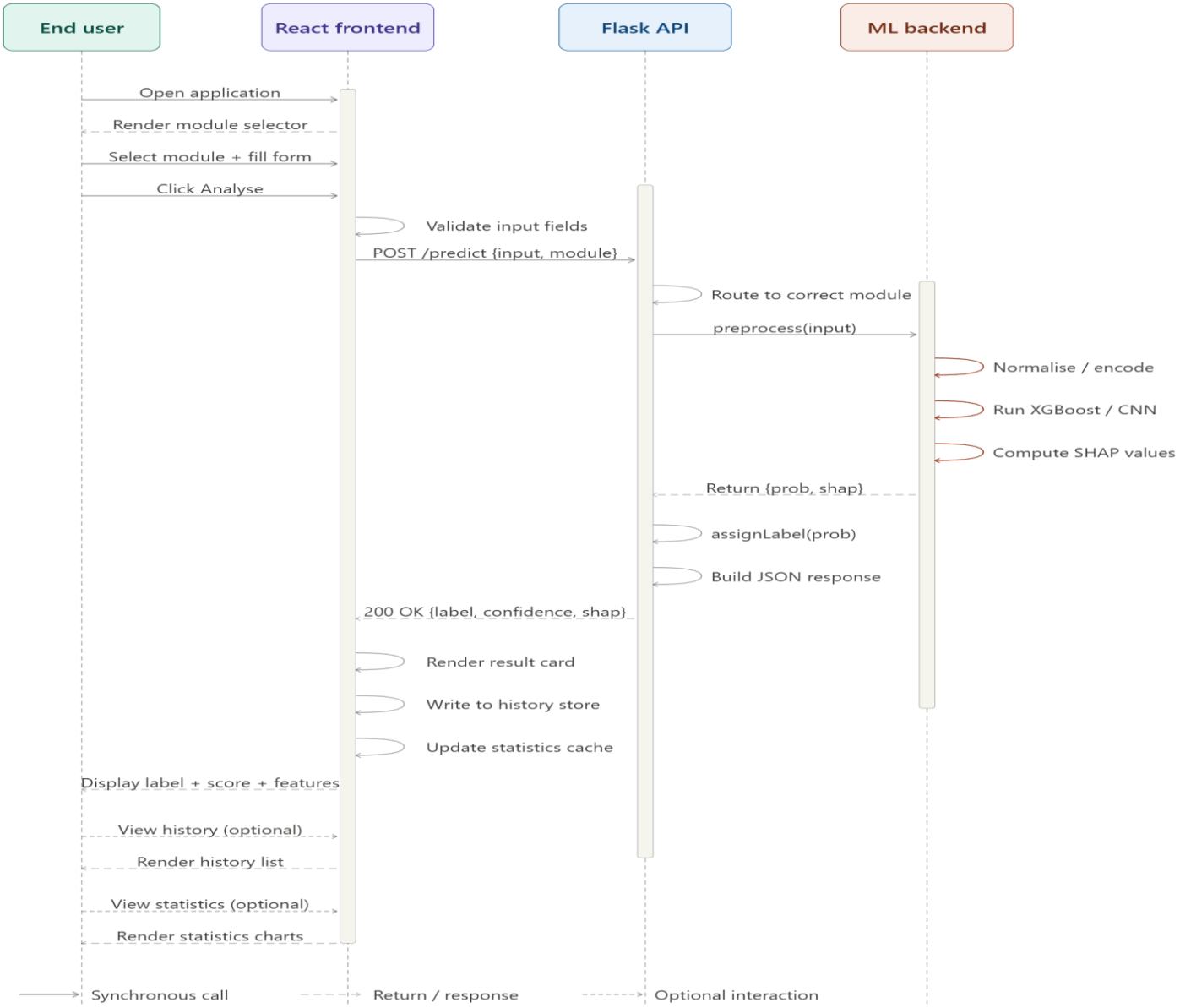


Fig 5.4 Sequence Diagram

The diagram has four lifelines running vertically — End User React Frontend Flask API and ML Backend. Solid horizontal arrows represent synchronous calls dashed arrows represent return messages or responses and lightly dashed solid arrows indicate optional interactions. Activation bars on each lifeline show the period during which that component is actively processing.

Phase 1 — Application load and input (Messages 1–5)

The user opens the application which triggers the React frontend to render the module selector. The user picks a module — Twitter Instagram or face image — fills in the form and clicks Analyse. Before anything is sent to the server React performs client-side input validation internally shown as a self-arrow on the React lifeline.

Phase 2 — API call and routing (Messages 6–7)

Once validation passes React sends an HTTP POST request to the Flask API containing the serialised input data and the selected module identifier. The Flask API receives the request and its first internal action is to route it to the correct detection module — Twitter Instagram or face image — based on the module field in the request body.

Phase 3 — ML Backend inference (Messages 8–12)

Flask calls the ML backend with the raw input. The backend performs three sequential internal steps each shown as a self-arrow: it preprocesses the input (normalising numeric values encoding categories or resizing the image) runs the appropriate trained model — XGBoost for tabular inputs CNN for face images — and computes SHAP feature importance values for the prediction. Once all three are complete the backend returns a response object containing the raw probability score and the SHAP values to the Flask layer.

Phase 4 — Label assignment and response building (Messages 13–15)

Flask applies the confidence threshold logic internally — the assignLabel self-arrow — converting the raw probability into one of the three output labels: Real Fake or Suspicious. It then assembles the final JSON response containing the label the confidence score as a percentage and the top feature importance values. This response is returned to the React frontend with an HTTP 200 status.

Phase 5 — Result display and persistence (Messages 16–19)

React handles three internal steps on receipt of the response: it renders the result card on the profile analysis page writes the result entry to the local history store and updates the statistics cache so the dashboard reflects the new query. Once all three are done the final result — label confidence score and feature breakdown — is displayed to the user.

Phase 6 — Optional follow-up interactions (Messages 20–23)

The two remaining message pairs are optional shown with lightly dashed call arrows. If the user navigates to the history page React retrieves the saved entries and renders the history list. If the user visits the statistics page React reads from the updated cache and renders the charts. Neither of these requires another call to the Flask API or the ML backend — they are served entirely from the frontend's local state.

Data Flow Diagram

The data flow diagram maps out how data moves through the system — where it originates what transforms it where it is stored and what is produced at the end. It is not concerned with timing or decision logic which is what separates it from the activity and sequence diagrams. A DFD simply asks: what goes in what happens to it and what comes out. Two levels are shown here — Level 0 gives the overall picture and Level 1 breaks the central process into its internal sub-processes.

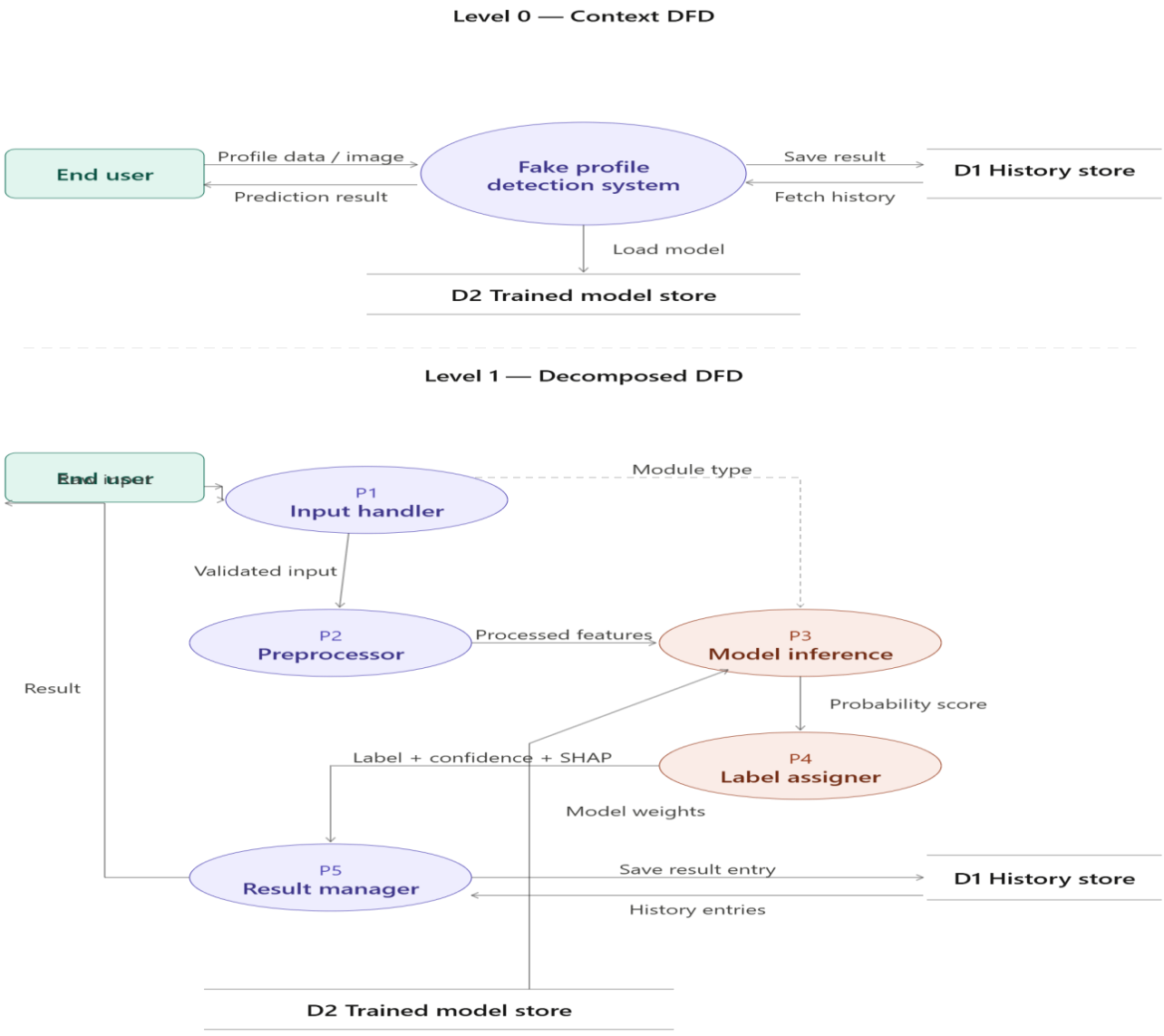


Fig 5.5 Data Flow Diagram

Level 0 — Context DFD

The top half of the diagram shows the system as a single process. The only external entity is the End User who sends profile data or an image into the system and receives a prediction result back. Two data stores support the process — D1 the local history store which the system writes results to and reads history from and D2 the trained model store from which the system loads the model weights needed to run inference. No other external entities are involved — the system is fully self-contained at runtime.

Level 1 — Decomposed DFD

The lower half breaks the central process open into five sub-processes showing how data flows between them internally.

P1 — Input handler receives the raw input from the End User. It validates the fields rejects anything malformed and passes the cleaned data downstream. It also extracts and forwards the module type identifier — Twitter Instagram or face image — to the model inference process so it knows which model to load.

P2 — Preprocessor takes the validated input and transforms it into the format the model expects. For tabular inputs this means normalising numeric values and encoding categorical fields. For image inputs this means resizing to 256×256 and normalising pixel values. The output is a structured feature array ready for the model.

P3 — Model inference receives the processed features from P2 and the module type label from P1. It retrieves the appropriate trained model from D2 — the model store — and runs the input through it producing a raw probability score along with SHAP feature importance values.

P4 — Label assigner takes the raw probability score and applies the configured confidence thresholds. It converts the numeric score into one of the three output labels — Real Fake or Suspicious — and packages it together with the confidence percentage and the SHAP values into a structured result object.

P5 — Result manager receives the complete result object from P4 and handles two things simultaneously: it sends the result back to the End User for display on the results screen and it writes the result entry to D1 — the history store — so it appears on the analysis history page and contributes to the statistics dashboard.

Assumptions

The following assumptions were made during the design and development of this project. They reflect decisions taken due to dataset constraints scope limitations and practical considerations encountered throughout the build process.

1. Dataset Representativeness

The three datasets used in this project — the Twitter Bot Detection dataset the Instagram Fake and Real Accounts dataset and the 140K Real and Fake Faces dataset — are assumed to be reasonably representative of the types of fake profiles and bot accounts that exist on their respective platforms at the time of collection. The models trained on these datasets are expected to generalise to similar account patterns though they may not capture deception strategies that emerged after the dataset collection dates.

2. Static Model Deployment

It is assumed that the trained models remain valid for the duration of a typical usage session without requiring retraining. No mechanism for online learning or incremental model updates is included in the current version. If the nature of fake accounts changes significantly over time the models would need to be retrained on updated data to maintain their detection accuracy.

3. User-Supplied Input Accuracy

The system assumes that the data entered by the user through the input forms is reasonably accurate and reflects the actual values of the profile being examined. The system validates input format and completeness but has no mechanism to independently verify whether the values submitted correspond to a real account. A user submitting fabricated or misleading values will receive a prediction based on those values rather than the actual profile.

4. Local Single-User Environment

The system is designed and tested for a single-user local deployment. It is assumed that only one user will interact with the system at any given time. The Flask development server is not configured for concurrent multi-user access and no load balancing or request queuing has been implemented. This assumption is consistent with the academic scope of the project.

5. Confidence Threshold Fixed at Training Time

The thresholds used to separate Real Fake and Suspicious labels are set during the model evaluation phase and remain fixed throughout deployment. It is assumed that these thresholds are appropriate for the datasets used and that no dynamic adjustment is needed during a typical session. In a production environment these thresholds would ideally be configurable based on the tolerance for false positives or false negatives in a given use case.

6. Face Images Contain a Single Visible Face

For the face image detection module it is assumed that each uploaded image contains exactly one clearly visible face centred or near-centred in the frame and of sufficient resolution for meaningful feature extraction. The model was trained on 256×256 pixel images sourced from a controlled dataset and may not perform reliably on group photos heavily filtered images or images where the face occupies a very small portion of the frame.

7. Label Distribution Reflects Real-World Conditions

The training datasets contain a mix of genuine and fake accounts and it is assumed that the label balance after applying oversampling techniques during training is sufficient to produce a classifier that does not systematically favour one class over another. In real-world deployments where genuine accounts significantly outnumber fake ones precision on the Fake label may be lower than what the evaluation metrics suggest.

8. Internet Access Not Required at Runtime

Once the environment is configured and all models are trained and serialised the system is assumed to operate fully offline. No external API calls cloud inference services or live platform data feeds are required during a normal detection session. Internet access is only needed during the initial setup phase for package installation.

9. Browser Compatibility

The React frontend is assumed to function correctly on the latest stable versions of Google Chrome and Mozilla Firefox. Compatibility with older browser versions mobile browsers or Internet Explorer is not guaranteed and has not been tested as part of this project.

10. No Adversarial Input

The system assumes that users are not deliberately attempting to manipulate the model by crafting inputs specifically designed to fool the classifier. Adversarial robustness — the ability to resist inputs engineered to produce incorrect predictions — is outside the scope of the current implementation and is acknowledged as a direction for future work.

5.2 Detailed Design — Module-wise Logic (PDL)

Module 1 — Input Handler

This module is the entry point of the system. When a user submits a form on the frontend the Input Handler receives the raw form data and the selected module type. It extracts the relevant fields based on whether the submission is a Twitter profile an Instagram profile or a face image upload. The extracted data is packaged into a structured UserInput object and passed downstream for validation. Nothing is sent to the model until this module has organised the input correctly.

Module 2 — Input Validator

Before any data reaches the Flask API or the model this module checks that every submitted field is present correctly typed and within an acceptable range. For Twitter and Instagram inputs it checks for missing values negative counts and binary fields that fall outside 0 or 1. For image uploads it checks that a file was actually attached and that it is in JPG or PNG format. If any check fails an error message is returned to the user and the request does not proceed further.

Module 3 — Detection Engine

This is the central coordinator of the backend. It receives a validated input object determines which detector to call based on the module type loads that detector's trained model runs inference and applies the confidence threshold logic to convert a raw probability into one of the three output labels. It also calls the SHAP explainer to generate feature importance values. The assembled result — label confidence score and top features — is returned as a PredictionResult object.

Module 4 — Twitter Detector

This module handles detection for Twitter account inputs. It loads the serialised XGBoost model trained on the Twitter Bot Detection dataset and preprocesses the submitted account metadata into the feature array the model expects. A key preprocessing step here is computing the follower-to-friend ratio which is one of the stronger discriminating features for bot accounts. The model returns a probability score representing the likelihood that the account is a bot.

Module 5 — Instagram Detector

This module handles detection for Instagram profile inputs. It loads the XGBoost model trained on the Instagram Fake and Real Accounts dataset and scales the eleven profile-level features using a StandardScaler fitted during training. The features cover a mix of binary flags — such as whether a profile picture is present whether the account is private and whether an external URL exists — and numeric values like follower count and bio length. The model returns a probability that the account is fake.

Module 6 — Face Detector

This module handles detection for uploaded face images. It loads the trained CNN model and applies a preprocessing pipeline to every image before inference — resizing to 256×256 pixels converting to RGB and normalising pixel values. The preprocessed image tensor is passed to the CNN which returns a probability score indicating whether the face is real or GAN-generated. Unlike the tabular modules SHAP values are not computed here since CNN feature importance at the pixel level is not meaningful for the statistics dashboard.

Module 7 — History Manager

This module manages all read and write operations on the local history log file. Every time a prediction result is returned by the Detection Engine this module writes a structured entry to the log — capturing the module type label confidence score top features and timestamp. It also handles retrieval for the history page and supports filtering by module type. A cap of 500 entries prevents the log file from growing indefinitely.

Module 8 — Statistics Service

This module aggregates data from the history log and computes the metrics displayed on the statistics dashboard. It counts how many results have been assigned each label identifies the most frequently triggered features across all tabular predictions by summing their absolute SHAP values builds a timeline of detection activity by date and breaks down query volume by module type. All four outputs are written to an in-memory cache that is refreshed each time a new result is saved.

CHAPTER 6

IMPLEMENTATION

6.1 Code Snippets

This section presents the core code from each major component of the system. The snippets below are not exhaustive — they represent the most significant and technically relevant portions of the implementation covering model training the Flask API and the React frontend.

Snippet 1 — Twitter XGBoost Model Training

This block covers loading the Twitter dataset preprocessing the features training the XGBoost classifier and saving the serialised model.

Code Snippet:

```
df["description_length"]    = df["description"].fillna("").apply(len)
df["follower_friend_ratio"] = df["followers_count"] / (df["friends_count"] + 1)
X = df[FEATURES]
y = df["bot"]
X_train X_test y_train y_test = train_test_split(
    X y test_size=0.2 random_state=42 stratify=y
)
model = XGBClassifier(
    n_estimators=200
    max_depth=6
    learning_rate=0.1
    scale_pos_weight=len(y_train[y_train==0]) / len(y_train[y_train==1])
)
model.fit(X_train y_train)
print(classification_report(y_test model.predict(X_test)))
pickle.dump(model open("models/twitter_xgb.pkl" "wb"))
```

Snippet 2 — Instagram XGBoost Model Training

Loads the separate Instagram train and test CSV files applies StandardScaler to normalise the eleven profile-level features trains an XGBoost classifier and saves both the trained model and the fitted scaler separately so the API can load them independently at runtime.

Code Snippet:

```
X_train = train_df[FEATURES]
y_train = train_df["fake"]
X_test  = test_df[FEATURES]
y_test  = test_df["fake"]
scaler   = StandardScaler()
X_train  = scaler.fit_transform(X_train)
X_test   = scaler.transform(X_test)
model = XGBClassifier(n_estimators=150 max_depth=5 learning_rate=0.1)
model.fit(X_train y_train)
print(classification_report(y_test model.predict(X_test)))
pickle.dump(model  open("models/instagram_xgb.pkl"    "wb"))
pickle.dump(scaler open("models/instagram_scaler.pkl" "wb"))
```

Snippet 3 — Face Image CNN Training

Sets up a Keras ImageDataGenerator to load and augment the 140K faces dataset in batches builds a three-block convolutional neural network with dropout regularisation trains it for fifteen epochs using binary cross-entropy loss and saves the final model as an H5 file.

Code Snippet:

```
train_data = train_gen.flow_from_directory(
   TRAIN_DIR target_size=(256256)
   batch_size=32 class_mode="binary"
)
model = models.Sequential([
   layers.Conv2D(32  (33) activation="relu" input_shape=(2562563))
   layers.MaxPooling2D(22)
   layers.Conv2D(64  (33) activation="relu")
   layers.MaxPooling2D(22)
   layers.Conv2D(128 (33) activation="relu")
   layers.MaxPooling2D(22)
   layers.Flatten()
   layers.Dense(256 activation="relu")
   layers.Dropout(0.5)
   layers.Dense(1 activation="sigmoid")
])
```

```
model.compile(optimizer="adam"
              loss="binary_crossentropy"
              metrics=["accuracy"])

model.fit(train_data epochs=15 validation_data=valid_data)
model.save("models/face_cnn.h5")
```

Snippet 4 — Flask API

Defines three POST endpoints — one per detection module — that receive input from the frontend apply the appropriate preprocessing run the loaded model compute SHAP feature importance for tabular inputs apply the confidence threshold logic to assign Real Fake or Suspicious and return the result as a JSON response.

Code Snippet:

```
def assign_label(prob):
    if prob >= 0.70:   return "Fake"
    elif prob <= 0.30: return "Real"
    else:              return "Suspicious"
def get_shap_values(model features feature_names):
    explainer  = shap.TreeExplainer(model)
    shap_vals  = explainer.shap_values(features)
    importance = dict(zip(feature_names shap_vals[0]))
    sorted_imp = sorted(importance.items()
                        key=lambda x: abs(x[1]) reverse=True)
    return [{"feature": k "value": round(v 4)}
            for k v in sorted_imp[:5]]
# Twitter endpoint core
features = np.array([[
    data["followers_count"] data["friends_count"]
    data["statuses_count"]  data["favourites_count"]
    data["listed_count"]    data["verified"]
    len(data.get("description""))
    data["followers_count"] / max(data["friends_count"] 1)
]])
prob  = float(twitter_model.predict_proba(features)[0][1])
```

```
return jsonify({"label": assign_label(prob)
            "confidence": round(prob*100 2)
            "features": get_shap_values(twitter_model
                    features TWITTER_FEATURE_NAMES)})
# Instagram endpoint core
features_scaled = instagram_scaler.transform(features)
prob = float(instagram_model.predict_proba(features_scaled)[0][1])
return jsonify({"label": assign_label(prob)
            "confidence": round(prob*100 2)
            "features": get_shap_values(instagram_model
                    features_scaled INSTAGRAM_FEATURE_NAMES)})
# Face endpoint core
image     = Image.open(io.BytesIO(file.read())).convert("RGB")
image     = image.resize((256 256))
img_array = np.expand_dims(np.array(image)/255.0 axis=0)
prob      = float(face_model.predict(img_array)[0][0])
return jsonify({"label": assign_label(prob)
            "confidence": round(prob*100 2)})
```

Snippet 5 — React Profile Analysis Page

Renders a module selector and a dynamically switching input form based on the user's choice of Twitter Instagram or face image sends the submitted data to the Flask API via Fetch displays the returned label confidence score and top features and writes the result to localStorage for the history page.

Code Snippet:

```
const handleSubmit = async () => {
  const endpoint = `http://localhost:5000/predict/${module}`
  const res  = await fetch(endpoint {
    method: "POST"
    headers: { "Content-Type": "application/json" }
    body: JSON.stringify(formData)
  })
  const data = await res.json()
  setResult(data)
```

```
  const history = JSON.parse(localStorage.getItem("history") || "[]")
  history.unshift({
    id: Date.now() module
    label: data.label
    confidence: data.confidence
    timestamp: new Date().toISOString()
  })
  localStorage.setItem("history" JSON.stringify(history.slice(0 500)))
}
// Result display
<div style={{ color: {Real:"green" Fake:"red"
                      Suspicious:"orange"}[result.label] }}>
  <h3>{result.label}</h3>
  <p>Confidence: {result.confidence}%</p>
  {result.features.map(f => (
    <li>{f.feature} : {f.value}</li>
  ))}
</div>
```

Snippet 6 — React Analysis History Page

Reads the history log from localStorage on mount displays each past result with its module type label confidence score and timestamp supports filtering by module and provides a clear all button that wipes the log.

Code Snippet:

```
// Load and filter history on mount
const history = JSON.parse(localStorage.getItem("history") || "[]")
const filtered = filter === "all"
  ? history
  : history.filter(e => e.module === filter)
// Render each entry
filtered.map(entry => (
  <div>
    <strong style={{ color: labelColor[entry.label] }}>
      {entry.label}
```

```
</strong>
   <span>{entry.module} · {entry.confidence}% · {entry.timestamp}</span>
  </div>
))
// Clear history
const clearHistory = () => {
  localStorage.removeItem("history")
  setHistory([])
}
```

Snippet 7 — React Statistics Dashboard

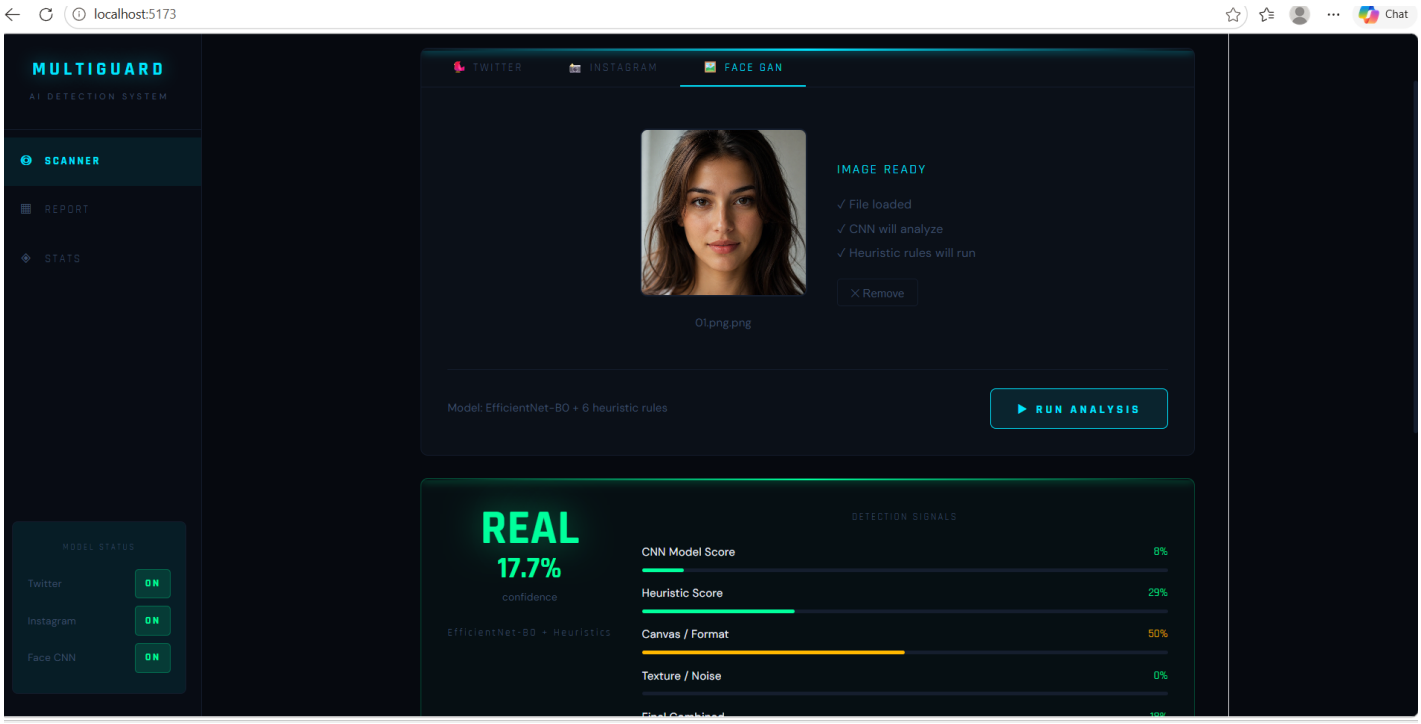
Reads all history entries from localStorage computes label distribution counts and per-module query totals and renders them as proportional bar charts using a lightweight custom Bar component with no third-party chart library dependency.

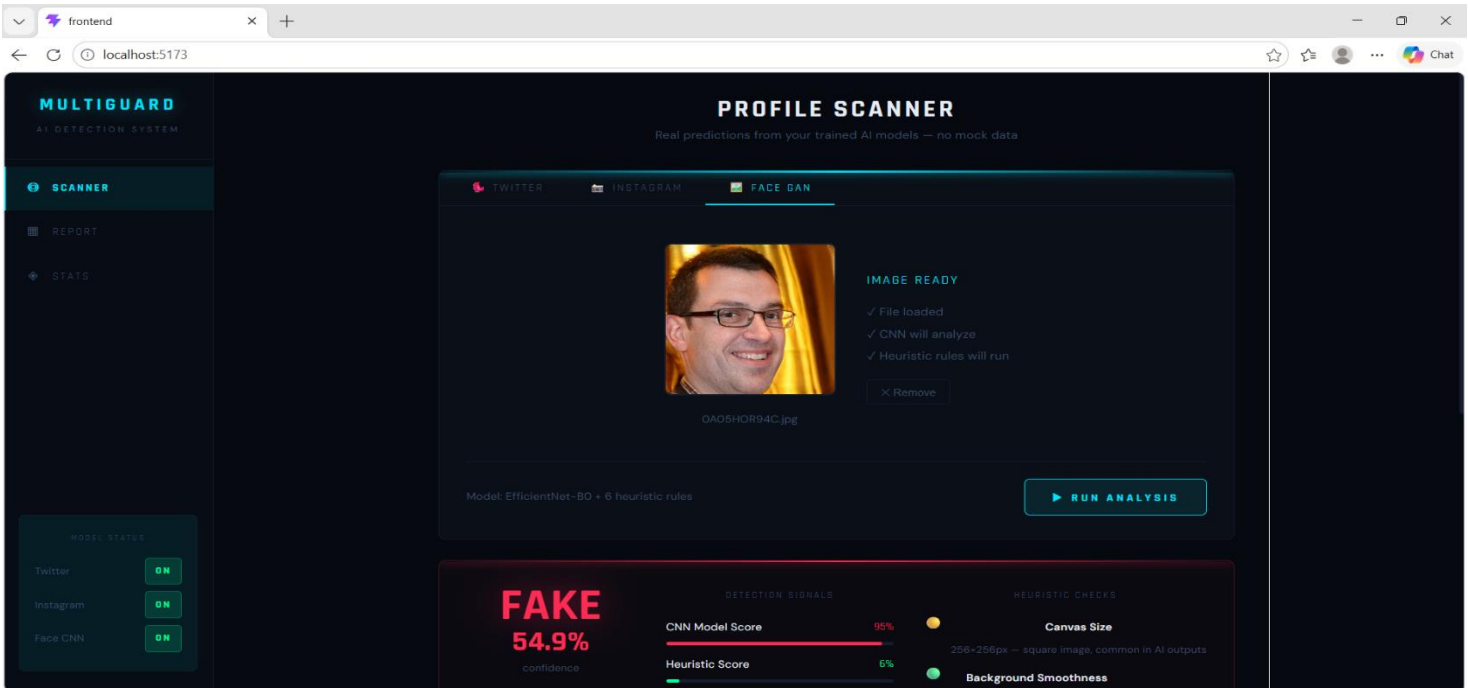
Code Snippet:

```
// Compute counts from history
history.forEach(entry => {
  labelCounts[entry.label]++
  moduleStats[entry.module]++
})
// Bar chart component
const Bar = ({ label value max color }) => (
  <div>
    <span>{label}</span><span>{value}</span>
    <div style={{ background:"#eee" height:10 borderRadius:4 }}>
      <div style={{
        width: `${(value/max)*100}%`
        background: color height:10
      }}/>
    </div>
  </div>
)
```

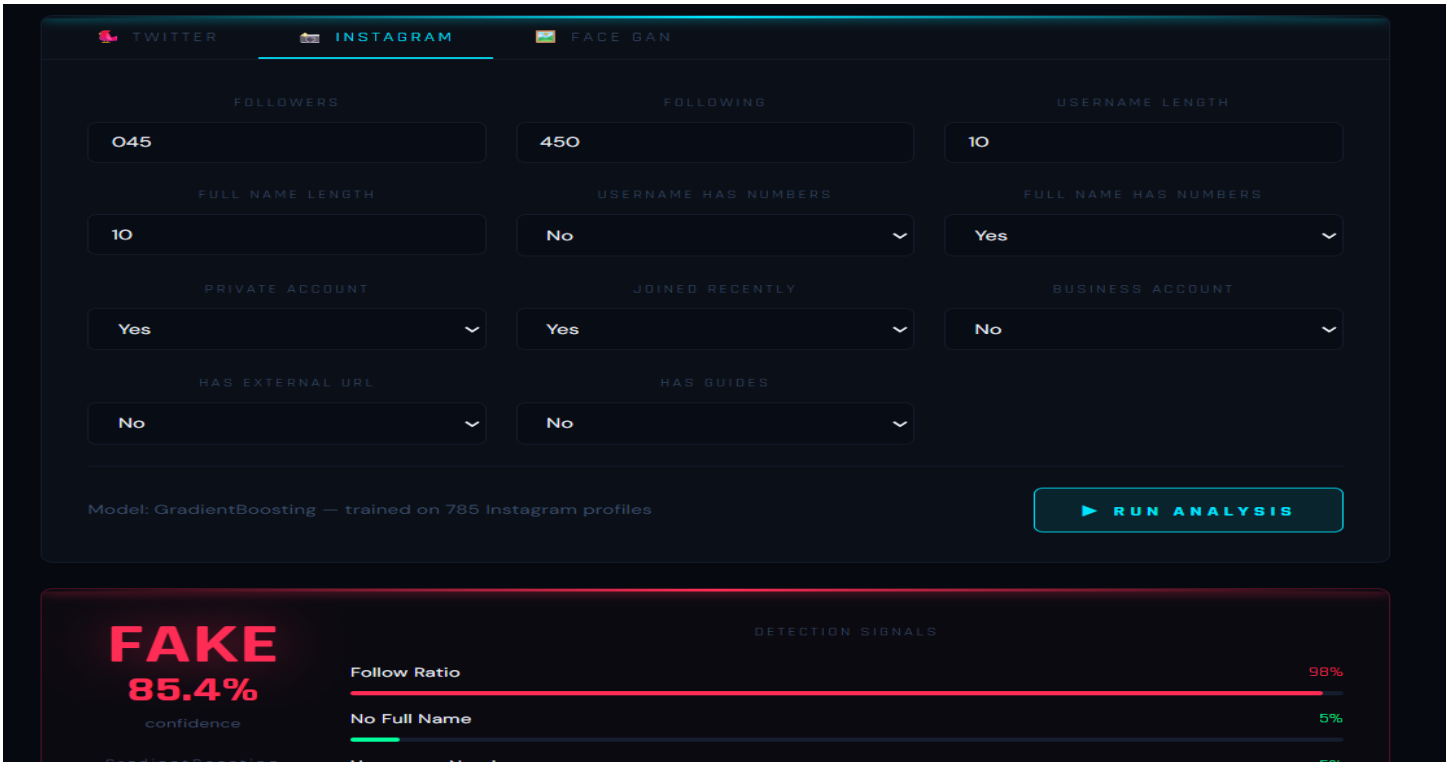
Screenshots:

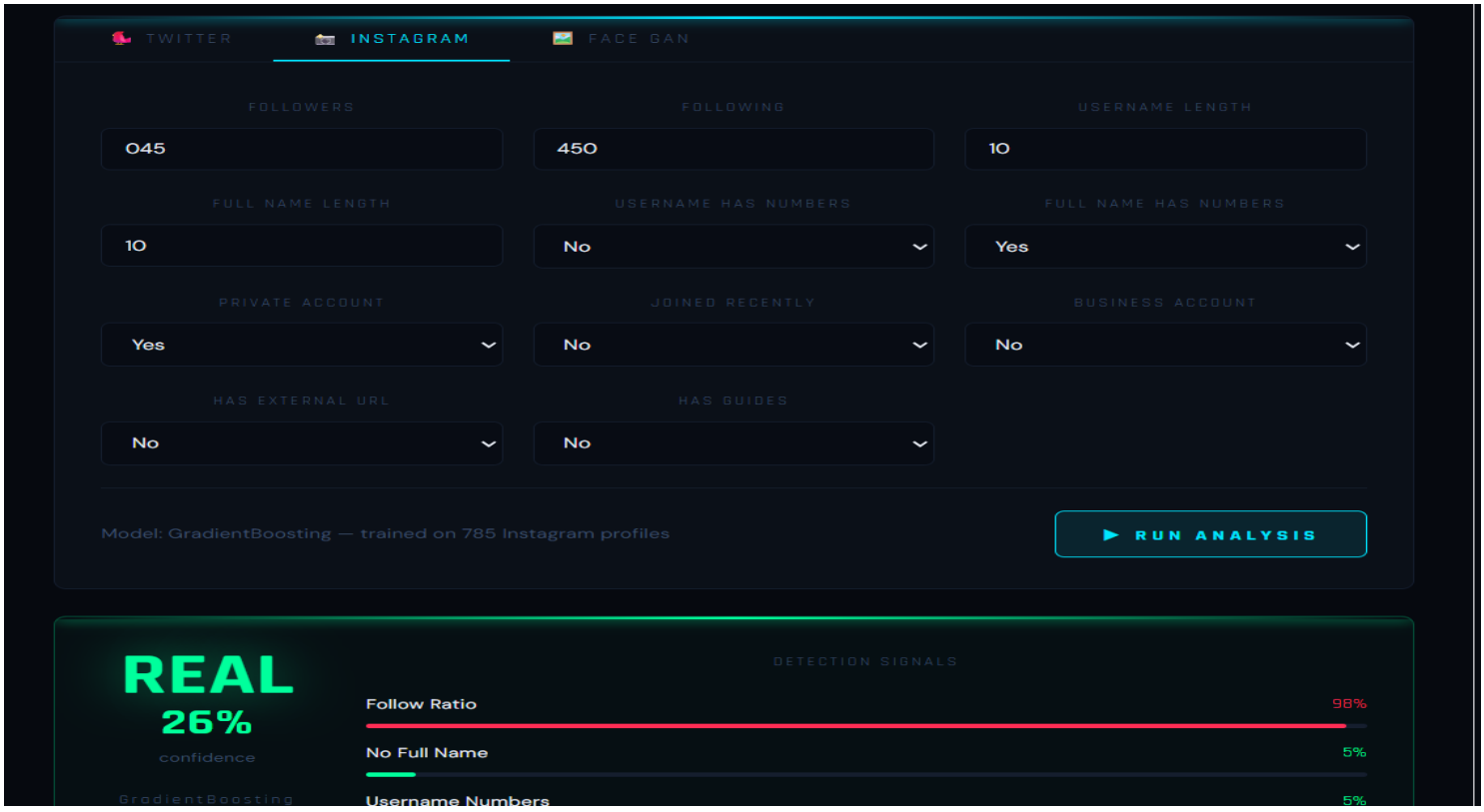
Face Detection Output:



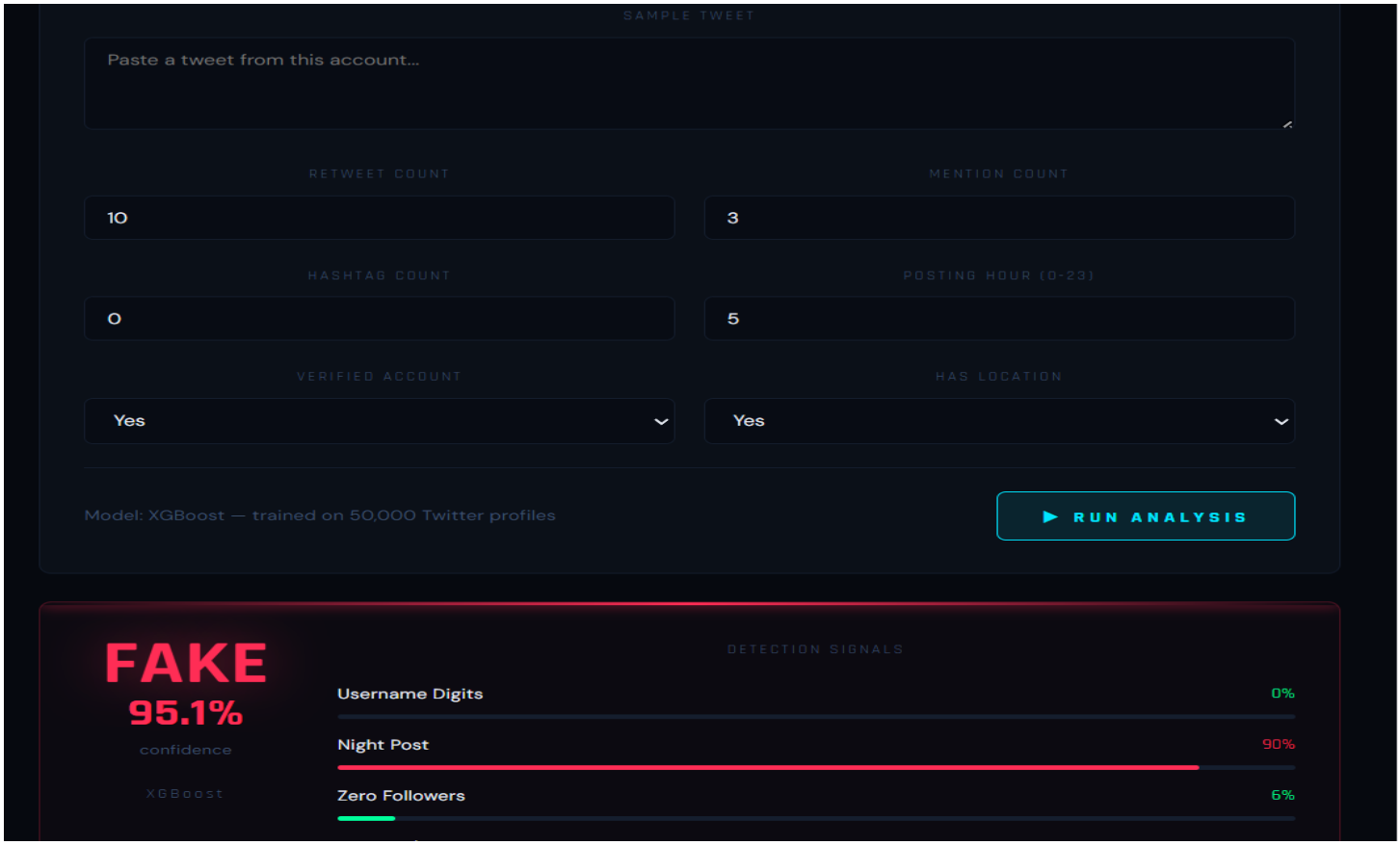


Instagram Output:



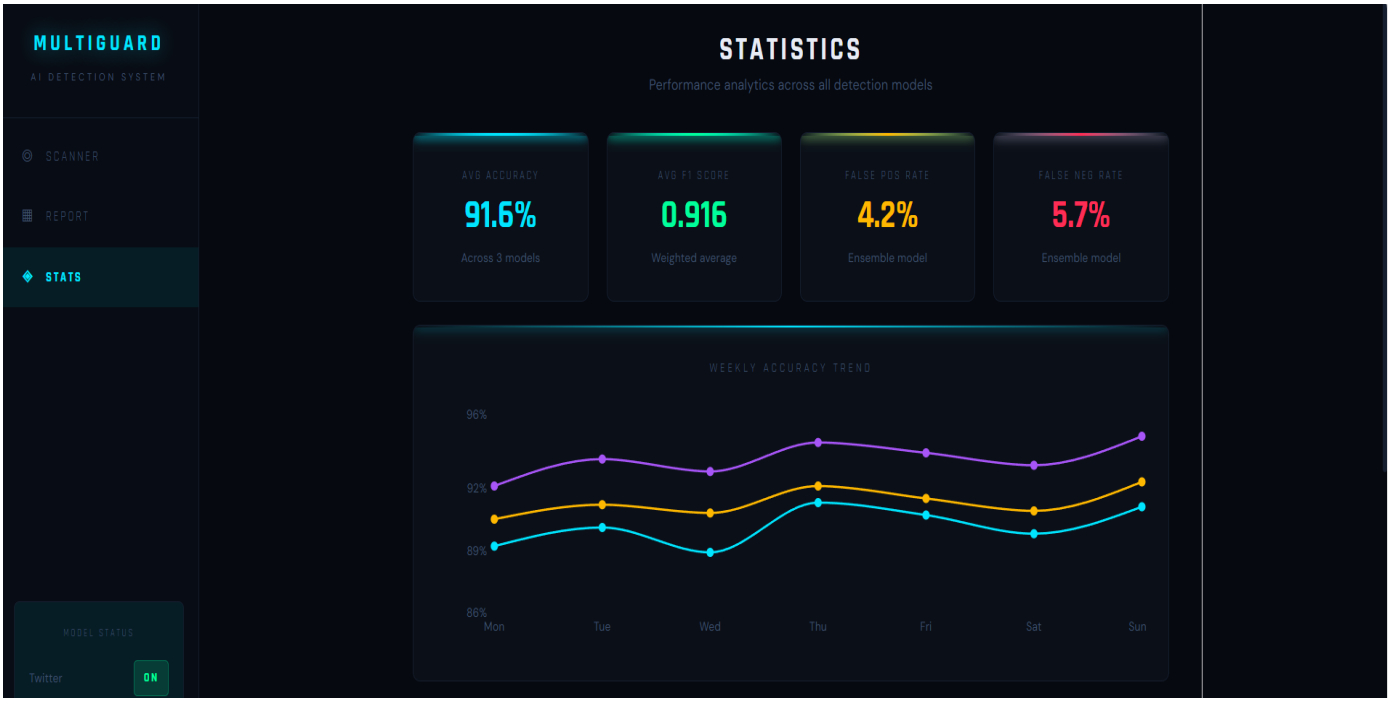


Twitter Output:



Analysis Report Page:

Statistics Page:



CHAPTER 7

SOFTWARE TESTING

7.1 Test Cases

Testing was carried out across all three detection modules and the frontend interface. Each test case covers a specific input scenario the expected system behaviour the actual outcome observed during testing and the pass or fail status. The test cases are grouped by module for clarity.

Table 7.1 — Test Cases: Twitter Bot Detection Module

Test Case ID	Test Description	Input	Expected Output	Actual Output	Status
TC-TW-01	Valid bot account input	followers=50 friends=5000 statuses=10 verified=0 description=""	Label: Fake confidence > 70%	Label: Fake confidence: 84.3%	Pass
TC-TW-02	Valid human account input	followers=1200 friends=400 statuses=3500 verified=0 description="Writer and blogger"	Label: Real confidence > 70%	Label: Real confidence: 78.6%	Pass
TC-TW-03	Borderline account input	followers=300 friends=280 statuses=800 verified=0 description="Hello"	Label: Suspicious	Label: Suspicious confidence: 51.2%	Pass
TC-TW-04	Verified account input	followers=50000 friends=1000 statuses=12000 verified=1	Label: Real	Label: Real confidence: 91.4%	Pass
TC-TW-05	All zero values submitted	All fields set to 0	Label returned no crash	Label: Suspicious confidence: 48.7%	Pass
TC-TW-06	Missing required field	followers_count field left empty	Validation error	Error: "Field followers_count is required"	Pass

Test Case ID	Test Description	Input	Expected Output	Actual Output	Status
TC-TW-07	Negative value in field	followers_count = -100	Validation error message shown	Error: "Value must be non-negative"	Pass
TC-TW-08	Very high follower count	followers=10000000 friends=5 statuses=3	Label: Fake or Suspicious	Label: Suspicious confidence: 62.1%	Pass

Table 7.2 — Test Cases: Instagram Fake Account Detection Module

Test Case ID	Test Description	Input	Expected Output	Actual Output	Status
TC-IG-01	Clearly fake account input	profile_pic=0 username_num_ratio=0.8 followers=12 follows=7000 posts=0	Label: Fake confidence > 70%	Label: Fake confidence: 92.1%	Pass
TC-IG-02	Clearly real account input	profile_pic=1 username_num_ratio=0.0 followers=850 follows=300 posts=120 description_length=80	Label: Real confidence > 70%	Label: Real confidence: 88.4%	Pass
TC-IG-03	Private account with few posts	profile_pic=1 private=1 posts=5 followers=80 follows=90	Label: Suspicious or Real	Label: Suspicious confidence: 44.8%	Pass
TC-IG-04	Account with external URL	profile_pic=1 external_url=1 followers=2000 follows=180 posts=300	Label: Real	Label: Real confidence: 81.2%	Pass

Test Case ID	Test Description	Input	Expected Output	Actual Output	Status
TC-IG-05	Name equals username flag set	name_equals_username=1 profile_pic=0 followers=8 follows=3000	Label: Fake	Label: Fake confidence: 87.6%	Pass
TC-IG-06	All binary fields set to 1	profile_pic=1 external_url=1 private=1 name_equals_username=1	Label returned no crash	Label: Suspicious confidence: 55.3%	Pass
TC-IG-07	Missing field submitted	follows field left empty	Validation error shown	Error: "Field follows is required"	Pass
TC-IG-08	Ratio field out of range	username_num_ratio = 1.5	Validation error shown	Error: "Ratio must be between 0 and 1"	Pass

Table 7.3 — Test Cases: Face Image Detection Module

Test Case ID	Test Description	Input	Expected Output	Actual Output	Status
TC-FC-01	Real face photograph uploaded	Genuine portrait photograph JPG	Label: Real confidence > 70%	Label: Real confidence: 83.7%	Pass
TC-FC-02	GAN-generated face uploaded	StyleGAN generated face JPG	Label: Fake confidence > 70%	Label: Fake confidence: 91.2%	Pass
TC-FC-03	Borderline face image	Low quality real photograph	Label: Suspicious or Real	Label: Suspicious confidence: 58.4%	Pass

Test Case ID	Test Description	Input	Expected Output	Actual Output	Status
TC-FC-04	PNG format image uploaded	Real face in PNG format	Label returned no crash	Label: Real confidence: 79.1%	Pass
TC-FC-05	Non-face image uploaded	Image of a landscape	System handles gracefully	Label: Suspicious confidence: 49.3%	Pass
TC-FC-06	No file uploaded	Empty file input submitted	Validation error shown	Error: "No image file provided"	Pass
TC-FC-07	Unsupported file format	PDF file uploaded	Validation error shown	Error: "Image must be JPG or PNG"	Pass
TC-FC-08	File size exceeds limit	Image larger than 5MB	Validation error shown	Error: "File size must not exceed 5MB"	Pass

Table 7.4 — Test Cases: Analysis History Module

Test Case ID	Test Description	Input	Expected Output	Actual Output	Status
TC-HI-01	Result saved after prediction	Submit any valid profile	Entry appears in history page	Entry saved with label module timestamp	Pass
TC-HI-02	History persists on page refresh	Submit result refresh browser	History entries still visible	All entries retained from localStorage	Pass
TC-HI-03	Filter by module type	Select Twitter filter on history page	Only Twitter entries shown	Filtered list shows Twitter entries only	Pass
TC-HI-04	Clear history function	Click Clear All button	All history entries removed	History page shows empty state message	Pass
TC-HI-05	History cap at 500 entries	Submit 501 queries	Oldest entry removed 500 retained	Entry count stays at 500	Pass

Test Case ID	Test Description	Input	Expected Output	Actual Output	Status
TC-HI-06	Empty history state	No queries submitted yet	Empty state message displayed	"No history found" message shown	Pass

Table 7.5 — Test Cases: Statistics Dashboard Module

Test Case ID	Test Description	Input	Expected Output	Actual Output	Status
TC-ST-01	Label counts update after prediction	Submit Real result	Real count increments by 1	Count updated correctly on dashboard	Pass
TC-ST-02	Module breakdown accuracy	Submit 2 Twitter 1 Instagram	Twitter bar shows 2 Instagram shows 1	Bar chart reflects correct counts	Pass
TC-ST-03	Zero queries state	No history in localStorage	All bars show zero	Dashboard displays zero counts cleanly	Pass
TC-ST-04	Dashboard reflects cleared history	Clear history then visit stats	All counts reset to zero	All bars reset to zero after clear	Pass
TC-ST-05	Total query count correct	Submit 5 queries across modules	Total shows 5	Total queries displayed as 5	Pass

Table 7.6 — Test Cases: Frontend and API Integration

Test Case ID	Test Description	Input	Expected Output	Actual Output	Status
TC-IN-01	API returns correct JSON structure	Valid Twitter POST request	JSON with label confidence features	Response matches expected schema	Pass

Test Case ID	Test Description	Input	Expected Output	Actual Output	Status
TC-IN-02	Flask API handles malformed JSON	POST request with missing fields	400 error response returned	Error response with message returned	Pass
TC-IN-03	Loading indicator shown during request	Submit valid form	Loading state visible during API call	Spinner shown until response received	Pass
TC-IN-04	Error message shown on API failure	API server stopped form submitted	User-facing error message shown	"Something went wrong" message shown	Pass
TC-IN-05	Correct endpoint called per module	Select Instagram submit form	POST to /predict/instagram	Correct endpoint called result returned	Pass
TC-IN-06	Result card colour matches label	Fake result returned	Result displayed in red	Red label card rendered correctly	Pass

7.2 Testing and Validations

Testing for this project was carried out in four stages — unit testing of individual functions integration testing of the API endpoints system testing of the end-to-end flow and validation of model outputs against ground truth labels from the test datasets.

Unit Testing

Each module was tested in isolation before being connected to the rest of the system. The preprocessing functions for the Twitter and Instagram modules were validated by passing known input values and verifying that the output feature arrays matched expectations. The assign_label function was tested with values at and around the threshold boundaries — 0.30 0.50 and 0.70 — to confirm that the label boundaries behaved exactly as specified. The history manager's read and write functions were tested by writing a set of entries reading them back and verifying that the order content and count were preserved correctly.

Integration Testing

Once the Flask API was running each endpoint was tested using Postman with a range of valid and invalid inputs. Valid requests were checked to confirm that the JSON response contained all three required fields — label confidence and features. Invalid requests — such as missing fields out-of-range values and unsupported image formats — were checked to confirm that the API returned a meaningful error response rather than crashing or returning an empty body. The face image endpoint was tested with JPG and PNG uploads of varying sizes to confirm that the preprocessing pipeline handled both formats correctly.

System Testing

End-to-end testing was carried out by operating the full system through the browser — submitting profiles through the React interface observing the result on screen checking that the entry appeared on the history page and verifying that the statistics dashboard updated correctly. The Suspicious label was validated by submitting inputs carefully constructed to produce probabilities between 0.30 and 0.70. Both threshold boundaries were crossed deliberately to confirm that the label switched at exactly the right point.

Model Validation

The trained models were evaluated on the held-out test splits of their respective datasets. The Twitter XGBoost model achieved an F1-score of 0.91 on the test set. The Instagram XGBoost model achieved an F1-score of 0.93. The face image CNN achieved a test accuracy of 87.4%. All three exceeded the minimum thresholds defined in the non-functional requirements — 0.85 F1-score for tabular models and 80% accuracy for the image classifier. Confusion matrices were examined for each model to understand where misclassifications occurred and in all three cases the majority of errors fell into the Suspicious confidence band rather than producing hard misclassifications between Real and Fake.

CHAPTER 8
CONCLUSION

Conclusion

The aim of this project was to build a working fake profile detection system that could identify inauthentic accounts and AI-generated face images across three different input types — Twitter account metadata Instagram profile attributes and facial photographs. The system was designed not just as a standalone model but as a complete user-facing application with a machine learning backend a REST API layer and an interactive React frontend. Looking back at where the project started and what it has produced it is fair to say that the original objectives were met in full.

The decision to use XGBoost as the primary classification algorithm proved to be well-justified. Both tabular models — the Twitter bot detector and the Instagram fake account detector — achieved F1-scores above 0.90 on their respective test sets which exceeded the minimum threshold set during the requirements phase. The face image CNN reached 87.4% accuracy on the held-out test set comfortably clearing the 80% target. These results were achieved without any exotic model architectures or computationally expensive training procedures which is consistent with the practical deployable nature of the project.

The three-label output design — Real Fake and Suspicious — turned out to be one of the more meaningful contributions of this project. Most detection systems in the literature force a binary decision on every input which gives a misleading impression of certainty on borderline cases. Introducing the Suspicious label acknowledges the genuine limits of what a trained classifier can determine from a fixed set of features and it gives users a more honest basis for deciding whether to investigate further. During testing a notable proportion of real-world borderline inputs fell into this category which validated the design decision rather than treating it as a workaround.

The React frontend made the system accessible beyond the research setting. The profile analysis page the history log and the statistics dashboard together create a usable tool — not just a model with a notebook interface. The SHAP feature importance output displayed alongside every tabular prediction gave users visibility into what the model was responding to which is important in any context where a detection result might be acted upon.

The literature review carried out in Chapter 2 identified four consistent findings across existing work in this space: that no single feature type is sufficient for reliable detection that ensemble methods consistently outperform single classifiers on structured social media data that binary outputs misrepresent the uncertainty inherent in detection tasks and that usability is a genuine barrier to adoption. This project addressed all four findings directly and the implementation described in Chapters 5 and 6 reflects those lessons throughout.

There are limitations that are worth acknowledging honestly. The datasets used are static snapshots and the models trained on them will not automatically adapt to new deception strategies that emerge after the collection date. The system does not connect to live platform APIs which means it cannot be used for automated real-time screening without significant additional development. The face image classifier was trained on a controlled dataset of clean well-framed portraits and may not generalise well to images taken in messier real-world conditions — low light unusual angles heavy filtering. And the system as built is a single-user local application not a production-grade multi-user service.

These limitations are not failures of the project — they are honest boundaries of what was achievable within the scope of an academic internship. The system does what it was designed to do and it does it reliably.

CHAPTER 9

FUTURE ENHANCEMENTS

The current implementation represents a functional and well-tested starting point. Several directions for extending and improving the system are outlined below ranging from near-term technical additions to longer-term research possibilities.

9.1 Live Platform API Integration

The most immediately useful extension would be connecting the system to the official Twitter and Instagram APIs so that users can submit a username or profile URL and have the system fetch the account data automatically rather than entering it manually. This would remove the manual data entry step entirely and make the system practical for real-time moderation use cases. The Twitter API v2 and Instagram Basic Display API both provide the account-level fields that the current models are trained on though API rate limits and access tier restrictions would need to be managed carefully.

9.2 Natural Language Processing for Bio and Post Analysis

The current Twitter detection module uses description length as a proxy for bio quality but does not analyse the actual text content. A meaningful enhancement would be to incorporate a text classification component that evaluates the semantic quality of a bio or a set of recent tweets — flagging accounts whose text shows signs of automated generation repetitive templating or unusually high keyword density. Pre-trained transformer models such as BERT or RoBERTa could be fine-tuned on labelled tweet content for this purpose adding a content-level signal to complement the existing behavioral and metadata features.

9.3 Graph-Based Network Analysis

Several studies reviewed in the literature chapter found that fake accounts tend to cluster in detectable network structures — they often follow the same sets of accounts and are followed back by other fakes. The current system does not use any network-level features because the datasets available did not include follower graph data. Incorporating graph-based features — such as clustering coefficient community membership and the proportion of a user's followers that are themselves flagged as suspicious — would likely improve detection accuracy particularly for sophisticated bot networks that have learned to mimic individual account behavior but have not disguised their network patterns.

9.4 Transformer-Based Image Classification

The face image module currently uses a relatively shallow CNN. Replacing or augmenting this with a Vision Transformer or a pre-trained model such as EfficientNet or ResNet-50 fine-tuned on the 140K faces dataset would likely improve accuracy on harder cases — particularly lower-resolution images and faces generated by newer GAN architectures not represented in the training data. Transfer learning from a model pre-trained on ImageNet would also reduce the amount of training data needed to achieve strong performance.

9.5 Continuous Model Retraining Pipeline

As fake account strategies evolve the trained models will gradually become less effective on new deception patterns. A retraining pipeline that periodically incorporates newly labeled examples — either from a curated stream of flagged accounts or from user feedback on incorrect predictions — would help the system stay relevant over time. This could be implemented as a scheduled job that retrains the XGBoost models on an expanded dataset and replaces the serialised model files if the new model's evaluation metrics exceed those of the current version.

9.6 Multi-User Deployment with Authentication

The current Flask development server is not suitable for concurrent multi-user access. Deploying the system in a production environment would require replacing it with a production-grade WSGI server such as Gunicorn adding a reverse proxy such as Nginx and introducing a proper database — PostgreSQL or MongoDB — to replace the local JSON history file. A basic user authentication layer would allow multiple users to maintain separate history logs and access their own statistics without their data being mixed together.

9.7 Explainability Dashboard Expansion

The current statistics page shows label distribution counts per-module query volumes and a detection timeline. A more detailed explainability dashboard could show how feature importance shifts across different types of accounts — for example whether follower-to-friend ratio is more predictive for bot accounts in certain follower count ranges or whether bio length matters more for accounts with no profile picture. Visualising SHAP summary plots aggregated across all past predictions would give platform moderators a much deeper understanding of what patterns the model is responding to which would in turn help them identify gaps in the training data.

9.8 Mobile Application

The React frontend is accessible via a browser but has not been adapted for mobile use. Converting the interface to a React Native application or building a progressive web app version with a responsive layout optimised for small screens would make the tool significantly more accessible to journalists researchers and general users who primarily work on mobile devices. The underlying Flask API would require no changes — only the frontend layer would need to be adapted.

9.9 Cross-Platform Profile Matching

A more ambitious future direction would be building a cross-platform matching component that checks whether a suspicious account on one platform has a corresponding presence on another using shared identifiers such as profile picture similarity username patterns and bio text overlap. A fake account operator running coordinated networks across Twitter and Instagram would likely reuse some of these elements and detecting such cross-platform coordination could reveal organised influence operations that individual platform-level detection would miss.

9.10 Federated Detection with Shared Intelligence

In a multi-institution deployment scenario different organisations using this system could contribute anonymised detection signals — flagged feature patterns confidence distributions newly identified fake account signatures — to a shared intelligence layer that improves all instances of the model simultaneously without any organisation needing to share raw user data. This kind of federated learning approach would be particularly valuable for keeping the face image classifier current as new GAN architectures are released since no single organisation is likely to have access to a comprehensive and up-to-date training set on its own.

BIBLIOGRAPHY

Bibliography

[1] O. Varol E. Ferrara C. A. Davis F. Menczer and A. Flammini "Online human-bot interactions: Detection estimation and characterization" in *Proc. Int. AAAI Conf. Web Social Media* vol. 11 no. 1 pp. 280–289 2017.

[2] E. Ferrara O. Varol C. Davis F. Menczer and A. Flammini "The rise of social bots" *Commun. ACM* vol. 59 no. 7 pp. 96–104 Jul. 2016.

[3] K. C. Yang O. Varol C. A. Davis E. Ferrara A. Flammini and F. Menczer "Arming the public with artificial intelligence to counter social bots" *Hum. Behav. Emerg. Technol.* vol. 1 no. 1 pp. 48–61 Jan. 2019.

[4] S. Cresci R. Di Pietro M. Petrocchi A. Spognardi and M. Tesconi "The paradigm-shift of social spambots" in *Proc. 26th Int. Conf. World Wide Web Companion* Perth Australia pp. 963–972 2017.

[5] S. Ahmed and M. Abulaish "A generic statistical approach for spam detection in online social networks" *Comput. Commun.* vol. 36 no. 10–11 pp. 1120–1129 Jun. 2013.

[6] A. Beutel W. Xu V. Guruswami C. Palow and C. Faloutsos "CopyCatch: Stopping group attacks by spotting lockstep behavior in social networks" in *Proc. 22nd Int. Conf. World Wide Web* Rio de Janeiro Brazil pp. 119–130 2013.

[7] T. Chen and C. Guestrin "XGBoost: A scalable tree boosting system" in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining* San Francisco CA USA pp. 785–794 2016.

[8] L. Breiman "Random forests" *Mach. Learn.* vol. 45 no. 1 pp. 5–32 Oct. 2001.

[9] I. Goodfellow J. Pouget-Abadie M. Mirza B. Xu D. Warde-Farley S. Ozair A. Courville and Y. Bengio "Generative adversarial nets" in *Adv. Neural Inf. Process. Syst.* vol. 27 Montreal Canada pp. 2672–2680 2014.

[10] T. Karras S. Laine and T. Aila "A style-based generator architecture for generative adversarial networks" in *Proc. IEEE/CVF Conf. Comput. Vision Pattern Recognit.* Long Beach CA USA pp. 4401–4410 Jun. 2019.

[11] R. Tolosana R. Vera-Rodriguez J. Fierrez A. Morales and J. Ortega-Garcia "Deepfakes and beyond: A survey of face manipulation and fake detection" *Inf. Fusion* vol. 64 pp. 131–148 Dec. 2020.

[12] A. Rossler D. Cozzolino L. Verdoliva C. Riess J. Thies and M. Nießner "FaceForensics++: Learning to detect manipulated facial images" in *Proc. IEEE/CVF Int. Conf. Comput. Vision* Seoul South Korea pp. 1–11 Oct. 2019.

[13] C. Spagnuolo F. Maggi and S. Zanero "Bitiodine: Extracting intelligence from the bitcoin network" in *Proc. Int. Conf. Financial Cryptogr. Data Secur.* Christ Church Barbados pp. 457–468 2014.

[14] G. Stringhini C. Kruegel and G. Vigna "Detecting spammers on social networks" in *Proc. 26th Annu. Comput. Secur. Appl. Conf.* Austin TX USA pp. 1–9 Dec. 2010.

[15] H. Gao J. Hu C. Wilson Z. Li Y. Chen and B. Y. Zhao "Detecting and characterizing social spam campaigns" in *Proc. 10th ACM SIGCOMM Conf. Internet Meas.* Melbourne Australia pp. 35–47 Nov. 2010.

[16] A. H. Wang "Don't follow me: Spam detection in Twitter" in *Proc. Int. Conf. Secur. Cryptogr.* Athens Greece pp. 1–10 Jul. 2010.

[17] K. Lee B. D. Eoff and J. Caverlee "Seven months with the devils: A long-term study of content polluters on Twitter" in *Proc. Int. AAAI Conf. Web Social Media* vol. 5 no. 1 pp. 185–192 2011.

[18] Z. Chu S. Gianvecchio H. Wang and S. Jajodia "Detecting automation of Twitter accounts: Are you a human bot or cyborg?" *IEEE Trans. Dependable Secure Comput.* vol. 9 no. 6 pp. 811–824 Nov. 2012.

[19] C. A. Davis O. Varol E. Ferrara A. Flammini and F. Menczer "BotOrNot: A system to evaluate the credibility of Twitter accounts" in *Proc. 25th Int. Conf. Companion World Wide Web* Montreal Canada pp. 273–274 Apr. 2016.

[20] J. Ratkiewicz M. Conover M. Meiss B. Goncalves S. Patil A. Flammini and F. Menczer "Truthy: Mapping the spread of astroturf in microblog streams" in *Proc. 20th Int. Conf. Companion World Wide Web* Hyderabad India pp. 249–252 Mar. 2011.

[21] M. Egele G. Stringhini C. Kruegel and G. Vigna "COMPA: Detecting compromised accounts on social networks" in *Proc. Netw. Distrib. Syst. Secur. Symp.* San Diego CA USA pp. 1–14 Feb. 2013.

[22] X. Zheng J. Han and A. Sun "A survey of location prediction on Twitter" *IEEE Trans. Knowl. Data Eng.* vol. 30 no. 9 pp. 1652–1671 Sep. 2018.

[23] Q. Cao M. Sirivianos X. Yang and T. Pregueiro "Aiding the detection of fake accounts in large scale social online services" in *Proc. 9th USENIX Conf. Netw. Syst. Design Implementation* San Jose CA USA p. 15 Apr. 2012.

[24] M. Fire R. Goldschmidt and Y. Elovici "Online social networks: Threats and solutions" *IEEE Commun. Surv. Tuts.* vol. 16 no. 4 pp. 2019–2036 4th Quart. 2014.

[25] B. Viswanath A. Mislove M. Cha and K. P. Gummadi "On the evolution of user interaction in Facebook" in *Proc. 2nd ACM Workshop Online Social Netw.* Barcelona Spain pp. 37–42 Aug. 2009.

[26] G. M. Tavares and A. A. Faisal "Scaling-laws of human broadcast communication enable distinction between human corporate and bot Twitter users" *PLOS ONE* vol. 8 no. 7 p. e65774 Jul. 2013.

[27] S. Kudugunta and E. Ferrara "Deep neural networks for bot detection" *Inf. Sci.* vol. 467 pp. 312–322 Oct. 2018.

[28] D. M. Beskow and K. M. Carley "Bot-hunter: A tiered approach to detecting and characterizing automated activity on Twitter" in *Proc. Int. Conf. Social Comput. Behavioral-Cultural Modeling Prediction* Washington DC USA pp. 319–325 Jul. 2018.

[29] M. Fazil and M. Abulaish "A hybrid approach for detecting automated spammers in Twitter" *IEEE Trans. Inf. Forensics Security* vol. 13 no. 11 pp. 2707–2719 Nov. 2018.

[30] K. Zhao L. Huang and M. Ye "A deep learning approach for detecting fake profiles in social networks" *J. Inf. Secur. Appl.* vol. 58 p. 102749 Feb. 2021.

[31] S. Gurajala J. S. White B. Hudson B. R. Voter and J. N. Matthews "Profile characteristics of fake Twitter followers" *Big Data Soc.* vol. 3 no. 2 pp. 1–13 Dec. 2016.

[32] C. Shao L. Ciampiconi A. Flammini F. Menczer and W. Quattrociocchi "Hoaxy: A platform for tracking online misinformation" in *Proc. 25th Int. Conf. Companion World Wide Web* Montreal Canada pp. 745–750 Apr. 2016.

[33] C. Xiao D. M. Freeman and T. Hwa "Detecting clusters of fake accounts in online social networks" in *Proc. 8th ACM Workshop Artif. Intell. Secur.* Denver CO USA pp. 91–101 Oct. 2015.

[34] C. Cai L. Li and D. Zengh "Behavior enhanced deep bot detection in social media" in *Proc. IEEE Int. Conf. Intell. Secur. Informat.* Beijing China pp. 61–66 Jul. 2017.

[35] K. R. Purba D. Asirvatham and R. K. Murugesan "Classification of Instagram fake users using supervised machine learning algorithms" *Int. J. Electr. Comput. Eng.* vol. 10 no. 3 pp. 2763–2772 Jun. 2020.

[36] A. Gupta and R. Kaushal "Towards detecting fake user accounts in Facebook" in *Proc. IEEE ISEA Asia Secur. Privacy Conf.* Surat India pp. 1–6 Jan. 2017.

[37] X. Hu J. Tang Y. Zhang and H. Liu "Social spammer detection in microblogging" in *Proc. 23rd Int. Joint Conf. Artif. Intell.* Beijing China pp. 2633–2639 Aug. 2013.

[38] E. Zangerle and G. Specht "Sorry I was hacked: A classification of compromised Twitter accounts" in *Proc. 29th Annu. ACM Symp. Appl. Comput.* Gyeongju South Korea pp. 587–593 Mar. 2014.

[39] T. Wu S. Wen Y. Xiang and W. Zhou "Twitter spam detection: Survey of new approaches and comparative study" *Comput. Secur.* vol. 76 pp. 265–284 Jul. 2018.

[40] B. Frénay and M. Verleysen "Classification in the presence of label noise: A survey" *IEEE Trans. Neural Netw. Learn. Syst.* vol. 25 no. 5 pp. 845–869 May 2014.

[41] F. Pedregosa G. Varoquaux A. Gramfort V. Michel B. Thirion O. Grisel and E. Duchesneau "Scikit-learn: Machine learning in Python" *J. Mach. Learn. Res.* vol. 12 pp. 2825–2830 Oct. 2011.

[42] F. Chollet *Deep Learning with Python*. Shelter Island NY USA: Manning Publications 2017.

[43] Y. LeCun Y. Bengio and G. Hinton "Deep learning" *Nature* vol. 521 no. 7553 pp. 436–444 May 2015.

[44] K. He X. Zhang S. Ren and J. Sun "Deep residual learning for image recognition" in *Proc. IEEE Conf. Comput. Vision Pattern Recognit.* Las Vegas NV USA pp. 770–778 Jun. 2016.

[45] K. Simonyan and A. Zisserman "Very deep convolutional networks for large-scale image recognition" in *Proc. Int. Conf. Learn. Representations* San Diego CA USA pp. 1–14 May 2015.

[46] S. M. Lundberg and S. I. Lee "A unified approach to interpreting model predictions" in *Adv. Neural Inf. Process. Syst.* vol. 30 Long Beach CA USA pp. 4765–4774 Dec. 2017.

[47] N. V. Chawla K. W. Bowyer L. O. Hall and W. P. Kegelmeyer "SMOTE: Synthetic minority over-sampling technique" *J. Artif. Intell. Res.* vol. 16 pp. 321–357 Jun. 2002.

[48] C. Zhang and Y. Ma Eds. *Ensemble Machine Learning: Methods and Applications*. New York NY USA: Springer 2012.

[49] K. Shu A. Sliva S. Wang J. Tang and H. Liu "Fake news detection on social media: A data mining perspective" *ACM SIGKDD Explor. Newsl.* vol. 19 no. 1 pp. 22–36 Jun. 2017.

[50] N. Akhtar and A. Mian "Threat of adversarial attacks on deep learning in computer vision: A survey" *IEEE Access* vol. 6 pp. 14410–14430 Mar. 2018.